Companion to Bird & de Moor, Algebra of Programming

§9. Dynamic Programming

§9.1. Theory

(9.1a)

	definition	type	meaning
1	`H ≜ (T)° (h)`	`A→B`	take the input apart by the `T`-decompositions, and put it together by `h`
2	`M ≜ H⅀ est(≤)`	`A→B`	the problem to be solved: an answer of `H` that is best under `≤`

§9.1.1. Stair example

(9.1.1a)

	symbol	value	meaning
1	`F`	`FX = 1 + (Move × X)` `Move ::= one(ℕ) | two(ℕ,ℕ)`	one layer of a climb: `1` ends it; `Move × X` takes one move, then `X` for the rest. `one c` stands on a stair of cost `c` and climbs 1; `two c d` stands on `c` and climbs 2, over the stair of cost `d`

2	`T`	`T : F[ℕ]→[ℕ]`
		`T ≜ [nil,put]`
		`nil = []`
		`put(one(c),xs) = [c]⧺xs`
		`put(two(c,d),xs) = [c]⧺[d]⧺xs`
			put the stairs of one move in front of the rest: `T(two 10 15, [20,30]) = [10,15,20,30]` stands on 10, climbs 2 over 15, and leaves `[20,30]` for the rest. `T°` splits `[10,15,20,30]` both ways a first move allows: `(one 10, [15,20,30])` and `(two 10 15, [20,30])`

3	`(T)`	`walk = (T)` `walk : [Move]→[ℕ]`	the cost list a climb walks over
4	`(T)°`	`[ℕ] → [Move]`	any climb over a cost list

5	`h`	`h ≜ [zero,pay]`
		`zero = 0`
		`pay(one(c),k) = c + k`
		`pay(two(c,d),k) = c + k`
			pay the stair stood on, not the `d` stepped over

6	`(h)`	`paid = (h)` `paid : [Move]→ℕ`	the total paid on a climb
7	`H`	`H = walk° paid`	the total paid on some climb over a cost list
8	`≤`	`≤` on `ℕ`	cheapest wins

§9.1.2. The dynamic programming theorem

Theorem 9.1

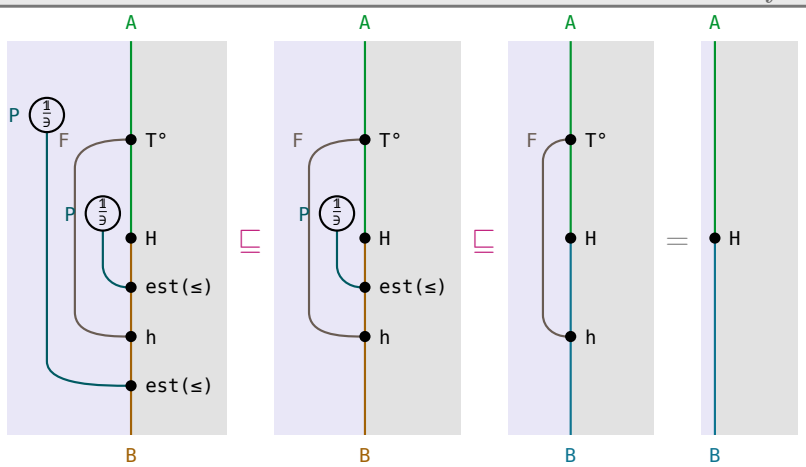
`Map(h); h Pres ≥; ≥² ⊑ ≥`
`⇒ (μX : T°⅀ P(F(X) h) est(≤)) ⊑ M`

every answer the recursion returns is an optimal one

- `T°⅀` takes the input apart one step every way; `F(X)` solves each part by the recursion `X`; `h` assembles each candidate; `est(≤)` keeps a best one
- `⊑M`: that never misses the optimum, though all candidates are never listed as `H` would
- `h Pres ≥`: the principle of optimality — a better part never makes the whole worse
- `⊑`, not `=`: some optima may not be returned; that the recursion returns anything at all needs `T°` to stop taking apart — dynamic programming theorem

$\frac{T°}{∋}$ P(F(M) h) est(≤) ⊑ H (9.2)

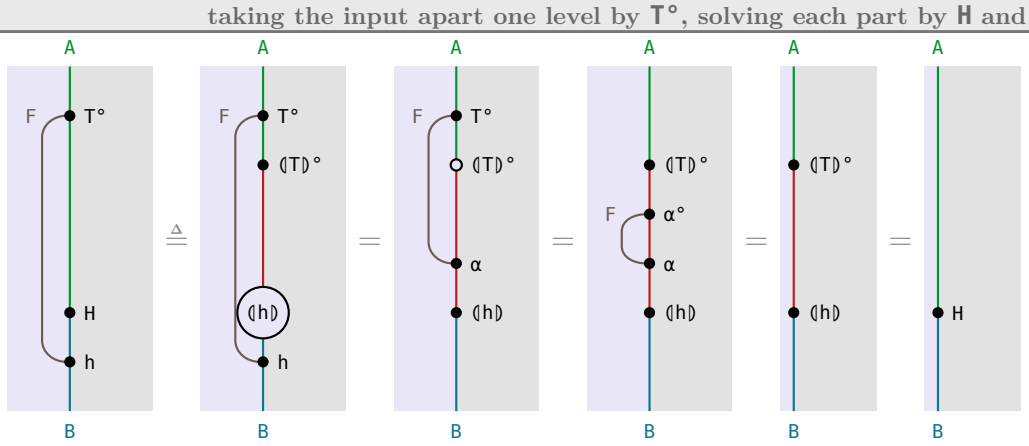
taking the input apart every way T allows ($\frac{T°}{∋}$), solving each part by M and keeping an optimum (P(F(M)h) est(≤)) returns only what H returns



1	$\frac{T°}{∋}$ P(F(M) h) est(≤)	$\frac{S}{∋}$ P(X) est(R) ⊑ S X
2	⊑ T° F(M) h	$\frac{H}{∋}$ est(≤) ⊑ H
3	⊑ T° F(H) h	(9.1.2a) T° F(H) h = H
4	= H	

T° F(H) h = H (9.1.2a)

taking the input apart one level by T°, solving each part by H and assembling by h gives H itself

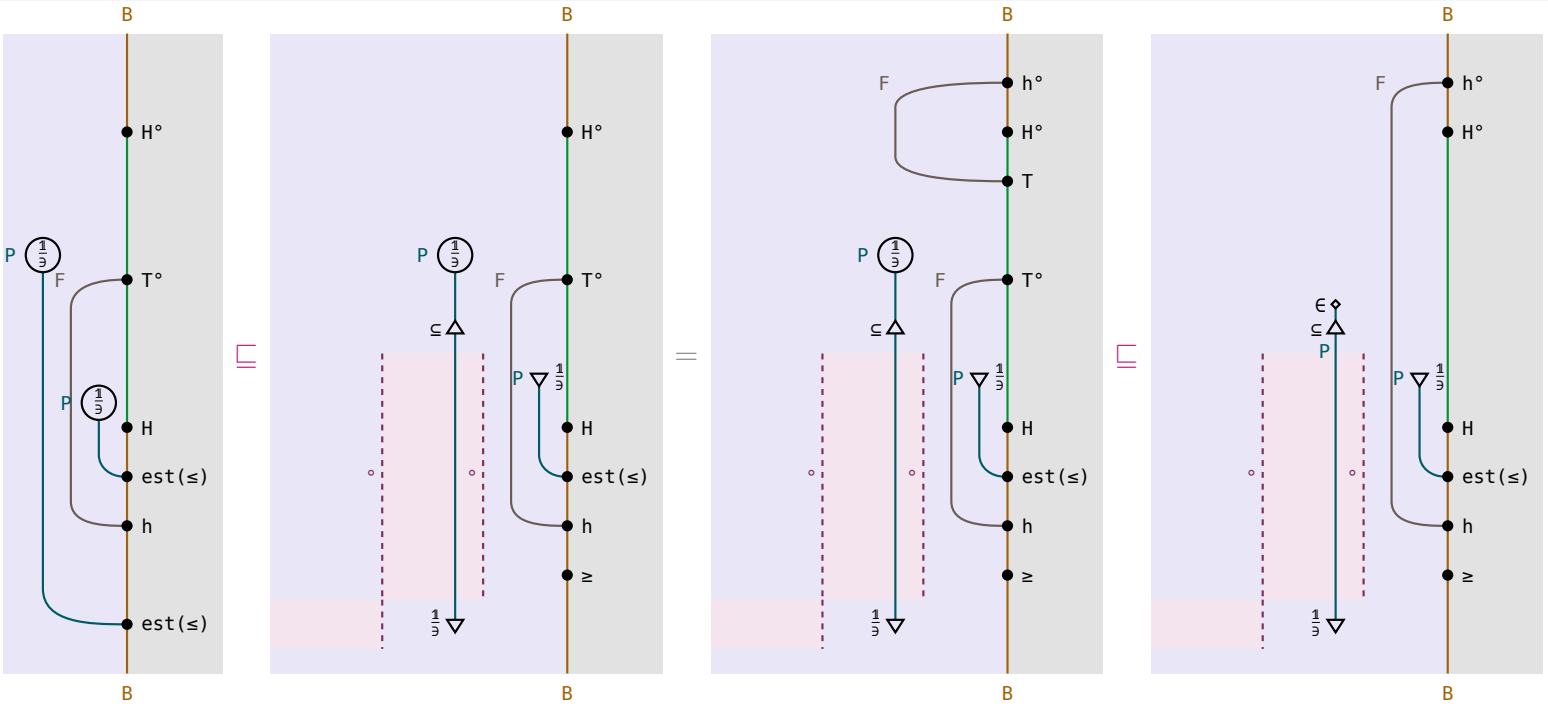


1	T° F(H) h	H ≜ (T)° (h)
2	≜ T° F((T)° (h)) h	α (h) = F((h)) h
3	= T° F((T)°) α (h)	(T)° α° = T° F((T)°)
4	= (T)° α° α (h)	α° α = 𝟙
5	= (T)° (h)	H = (T)° (h)
6	= H	

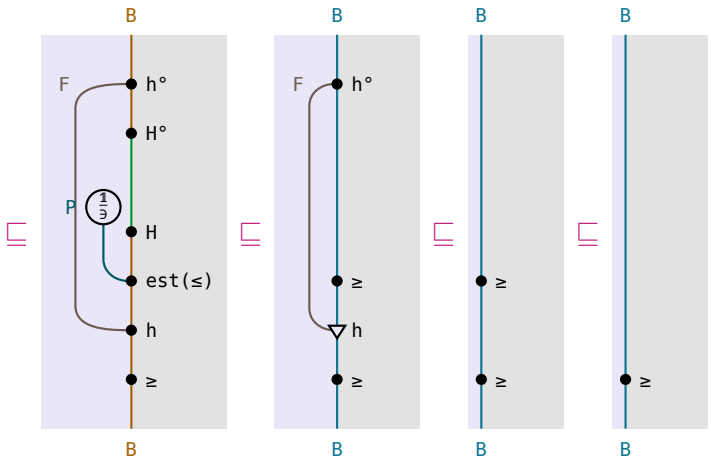
Map(h); h Pres ≥; ≥² ⊑ ≥ **(9.3)**

⇒ H° $\frac{T°}{∋}$ P(F(M) h) est(≤) ⊑ ≥

for every **b** that **H** returns from an input, the step $\frac{T°}{∋}$ **P(F(M)h) est(≤)** returns from that input only **b'** with **b' ≤ b**



1	H° $\frac{T°}{∋}$ P(F($\frac{H}{∋}$ est(≤)) h) est(≤)	P(X) est(R) ⊑ ∈ \ (X R°)
2	⊑ H° $\frac{T°}{∋}$ (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	H° = h° F(H°) T
3	= h° F(H°) T $\frac{T°}{∋}$ (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	T $\frac{T°}{∋}$ ⊑ ∈
4	⊑ h° F(H°) ∈ (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	∈ (∈ \ X) ⊑ X

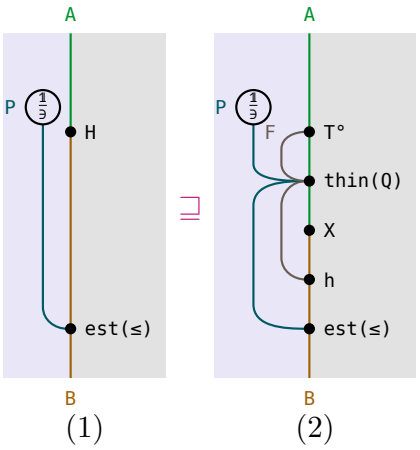


5	⊑ h° F(H° $\frac{H}{∋}$ est(≤)) h ≥	H° $\frac{H}{∋}$ est(≤) ⊑ ≥
6	⊑ h° F(≥) h ≥	Map(h) ⇒ h Pres ≥ ⇔ h° F(≥) h ⊑ ≥
7	⊑ ≥²	≥² ⊑ ≥
8	⊑ ≥	

Map(h); h Pres ≥; ≥² ⊑ ≥; ThinCondition(T,h,≤,Q)
⇒ (μX : $\frac{T°}{∋}$ thin(Q) P(F(X) h) est(≤)) ⊑ M — Theorem 9.2

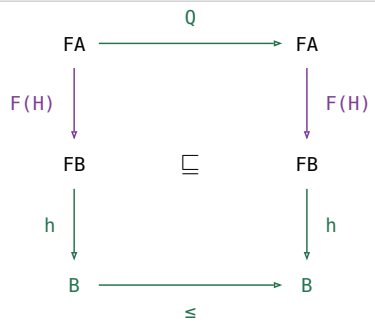
an optimum over everything H returns is reached by taking the input apart every way T allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

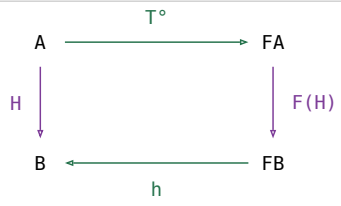
H≜(T)°(h) — H ≜ (T)° (h)



(1) (2)

1	M	ThinCondition(T,h,≤,Q) ≜ Q F(H) h ⊑ F(H) h ≤ , h monotonic on ≤ — Q ⊑ R; 𝟙 ⊑ Q; R² ⊑ R ⇒ est(R) = thin(Q) est(R) , X ⊑ est(R) ⇔ X ⊑ ∋ ∧ ∈ X ⊑ R°
2	⊒ $\frac{T°}{∋}$ thin(Q) P(F(X) h) est(≤)	



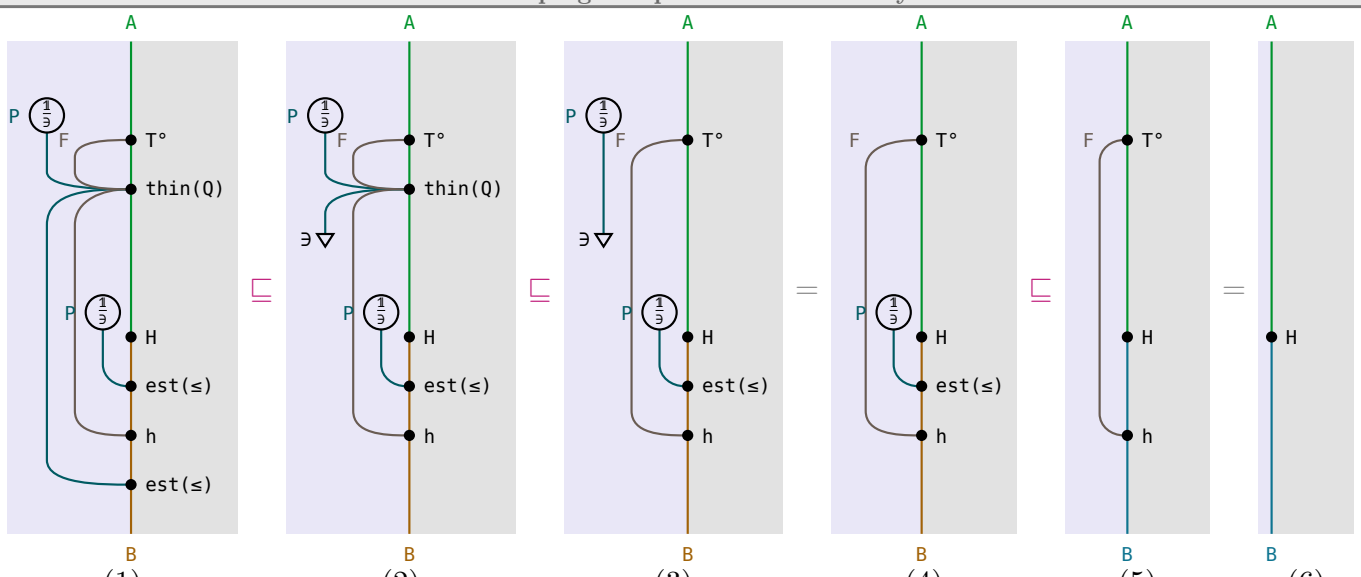


T°F(H)h=H: taking the input apart once by T°, solving the parts by F(H) and assembling by h returns exactly what H returns

QF(H)h ⊑ F(H)h ≤: every result of thinning by Q, solving the parts by F(H) and assembling by h is reached by ≤ from a result of solving by F(H) and assembling by h without Q

$\frac{T°}{∋}$ thin(Q) P(F(M) h) est(≤) ⊑ H — (9.1.2b)

taking the input apart every way T allows ($\frac{T°}{∋}$), dropping the parts thin(Q) rejects, solving each rest by M and keeping an optimum returns only what H returns

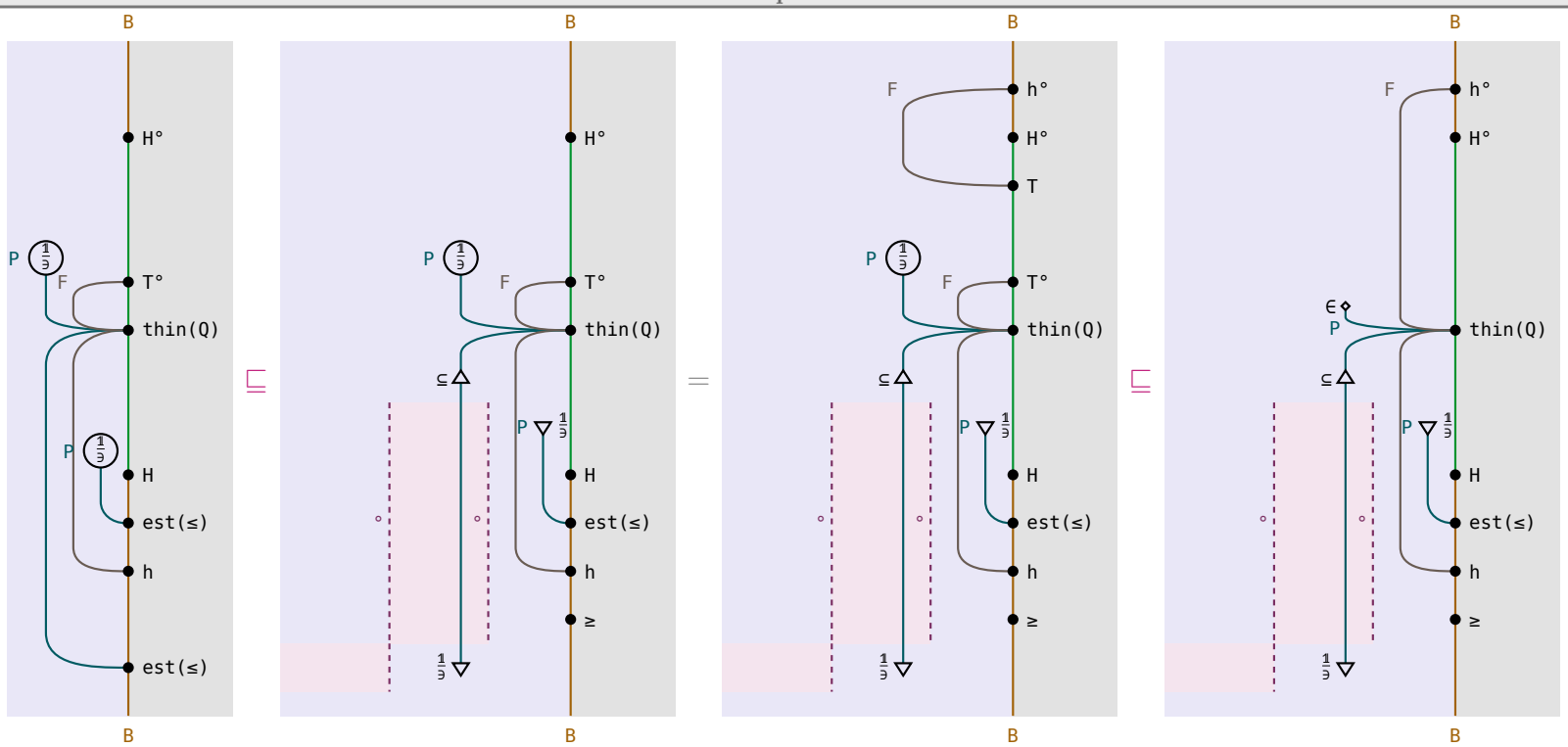


1	$\frac{T°}{∋}$ thin(Q) P(F($\frac{H}{∋}$ est(≤)) h) est(≤)	P(X) est(R) ⊑ ∋ X
2	⊑ $\frac{T°}{∋}$ thin(Q) ∋ F($\frac{H}{∋}$ est(≤)) h	thin(Q) ∋ ⊑ ∋
3	⊑ $\frac{T°}{∋}$ ∋ F($\frac{H}{∋}$ est(≤)) h	$\frac{R}{∋}$ ∋ = R
4	= T° F($\frac{H}{∋}$ est(≤)) h	$\frac{H}{∋}$ est(≤) ⊑ H
5	⊑ T° F(H) h	(9.1.2a) T° F(H) h = H
6	= H	

Map(h); h Pres ≥; ≥² ⊑ ≥; ThinCondition(T,h,≤,Q) (9.1.2c)

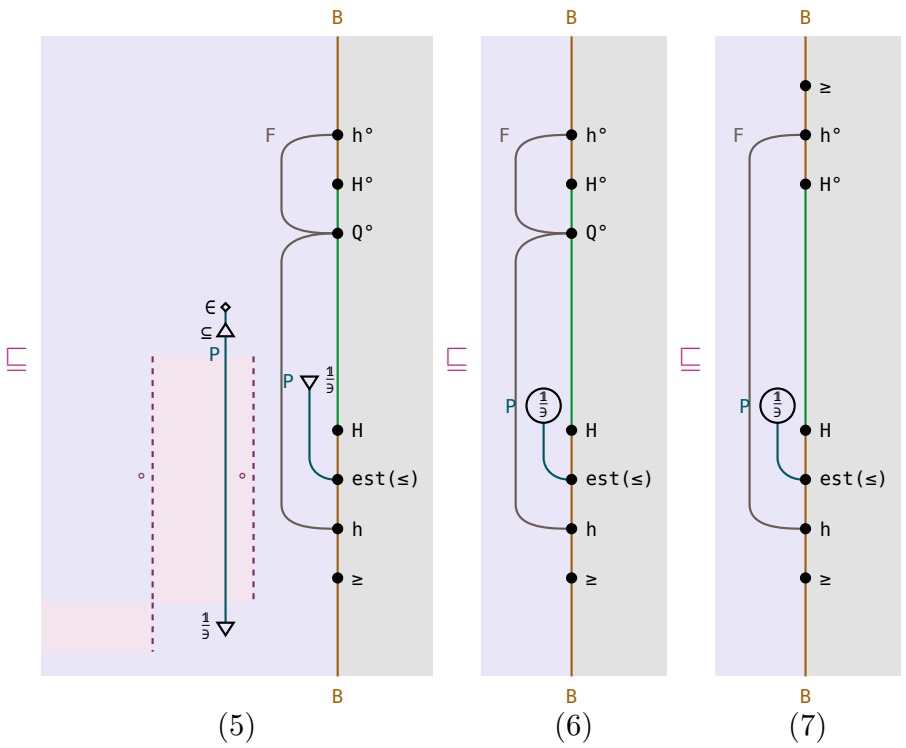
⇒ H° $\frac{T°}{∋}$ thin(Q) P(F(M) h) est(≤) ⊑ ≥

H° followed by the body at **M** is **⊑≥**: an answer of the body is never worse than an answer of **H** to the same input



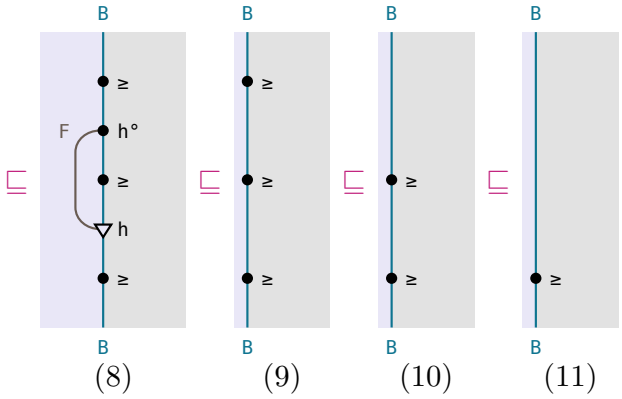
(1) (2) (3) (4)

1	H° $\frac{T°}{∋}$ thin(Q) P(F($\frac{H}{∋}$ est(≤)) h) est(≤)	P(X) est(R) ⊑ ∈ \ (X R°)
2	⊑ H° $\frac{T°}{∋}$ thin(Q) (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	H° = h° F(H°) T
3	= h° F(H°) T $\frac{T°}{∋}$ thin(Q) (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	T $\frac{T°}{∋}$ ⊑ ∈
4	⊑ h° F(H°) ∈ thin(Q) (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	∈ thin(Q) ⊑ Q° ∈



(5) (6) (7)

5	⊑ h° F(H°) Q° ∈ (∈ \ (F($\frac{H}{∋}$ est(≤)) h ≥))	∈ (∈ \ X) ⊑ X
6	⊑ h° F(H°) Q° F($\frac{H}{∋}$ est(≤)) h ≥	ThinCondition(T,h,≤,Q) ⇒ h° F(H°) Q° ⊑ ≥ h° F(H°)
7	⊑ ≥ h° F(H° $\frac{H}{∋}$ est(≤)) h ≥	H° $\frac{H}{∋}$ est(≤) ⊑ ≥



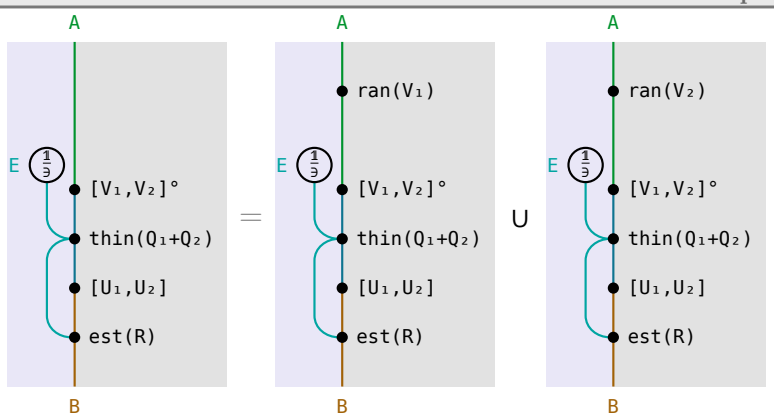
(8) (9) (10) (11)

8	$\sqsubseteq \geq$ h° F(≥) h ≥	Map(h) ⇒ h Pres ≥ ⇔ h° F(≥) h ⊑ ≥
9	$\sqsubseteq \geq^3$	$\geq^2$ ⊑ ≥
10	$\sqsubseteq \geq^2$	$\geq^2$ ⊑ ≥
11	$\sqsubseteq \geq$	

V_2 V_1° = ∅; V_1 V_2° = ∅ **Proposition 9.1**

⇒ $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R) = (ran(V_1) $\frac{V_1^\circ}{\ni}$ thin(Q_1) P(U_1) est(R))
∪ (ran(V_2) $\frac{V_2^\circ}{\ni}$ thin(Q_2) P(U_2) est(R))

when V_1 and V_2 have disjoint ranges, the body over [V_1,V_2] runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches

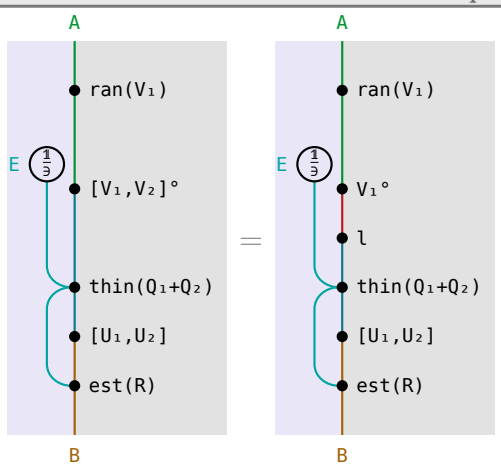


1	$\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R)	$\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R) = (ran(V_1) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R)) ∪ (ran(V_2) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R))
2	= (ran(V_1) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R)) ∪ (ran(V_2) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q_1 + Q_2) P([U_1,U_2]) est(R))	

V₂ V₁° = 𝟘 (9.1.2d)

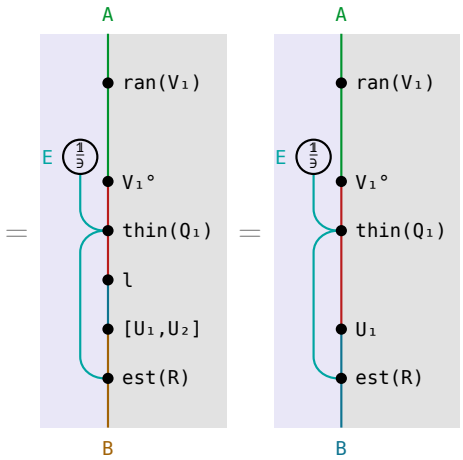
⇒ ran(V₁) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q₁ + Q₂) P([U₁,U₂]) est(R) = ran(V₁) $\frac{V_1^\circ}{\ni}$ thin(Q₁) P(U₁) est(R)

on inputs V₁ reaches, the body over [V₁,V₂] is the V₁ problem



(1) (2)

1	ran(V₁) $\frac{[V_1,V_2]^\circ}{\ni}$ thin(Q₁ + Q₂) P([U₁,U₂]) est(R)	V₂ V₁° = 𝟘 ⇒ ran(V₁) $\frac{[V_1,V_2]^\circ}{\ni}$ = ran(V₁) $\frac{V_1^\circ}{\ni}$ P(l)
2	= ran(V₁) $\frac{V_1^\circ}{\ni}$ P(l) thin(Q₁ + Q₂) P([U₁,U₂]) est(R)	P(l) thin(Q₁ + Q₂) = thin(Q₁) P(l)



(3) (4)

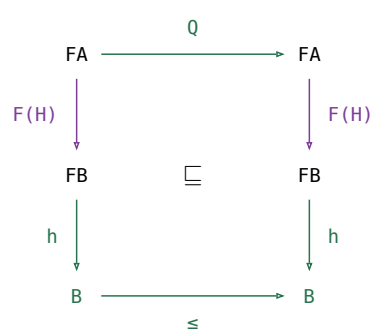
3	= ran(V₁) $\frac{V_1^\circ}{\ni}$ thin(Q₁) P(l [U₁,U₂]) est(R)	l [U₁,U₂] = U₁
4	= ran(V₁) $\frac{V_1^\circ}{\ni}$ thin(Q₁) P(U₁) est(R)	

§9.1.3. Checking the conditions

(9.1.3a)

	definition	**meaning**
1	H ≜ ⦇T⦈° ⦇h⦈	take the input apart by the T-decompositions, and put it together by h
2	ThinCondition(T,h,≤,Q) ≜ Q F(H) h ⊑ F(H) h ≤	thinning the decompositions by Q before solving them only moves the answer up ≤
	2 conditions of DP	**what it says**
1	h Pres ≥	xs −F(≥)→ ys −h→ Y ⟹ xs −h→ X −≥→ Y for some X (h brings the parts to the whole)
2	ThinCondition(T,h,≤,Q)	thinning the decompositions by Q loses no optimal answer

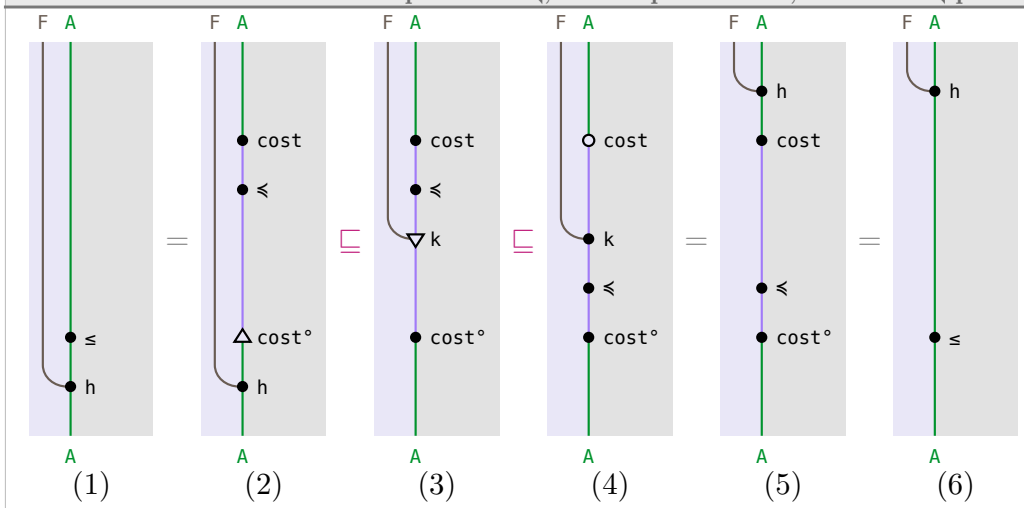
(9.1.3b)



every result of thinning by Q, solving the parts by F(H) and assembling by h is reached by ≤ from a result of solving by F(H) and assembling by h without Q: dropping decompositions never loses an answer that is not matched by a kept one at least as good

Map(cost); ≤ = cost ≼ cost°; (cost : h → k); k Pres ≼ ⇒ h Pres ≤ — Proposition 9.2

an F-algebra homomorphism `cost : h → k` carries preservation back from `k` to `h`: if `k` preserves ≼, then `h` preserves ≤, the order ≼ pulled back along `cost`



(1) (2) (3) (4) (5) (6)

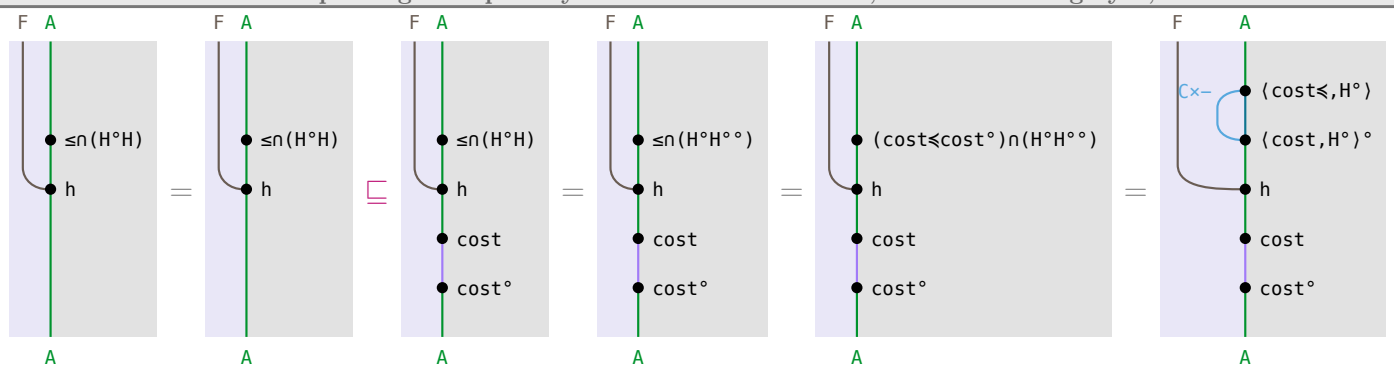
1	`F(≤) h`	`≤ = cost ≼ cost°`
2	`= F(cost) F(≼) F(cost°) h`	`Map(cost); (cost : h → k) ⇒ F(cost°) h ⊑ k cost°`
3	`⊑ F(cost) F(≼) k cost°`	`k Pres ≼`
4	`⊑ F(cost) k ≼ cost°`	`(cost : h → k)`
5	`= h cost ≼ cost°`	`≤ = cost ≼ cost°`
6	`= h ≤`	

(9.1.3c)

	symbol	value	meaning
1	`F`	`F(X) = [X]`	the list functor
2	`h`	`concat : [String] → String`	joins the pieces into one string
3	`cost`	`length : String → ℕ`	string length
4	`≼`	`≤` on `ℕ`	compares the costs
5	`k`	`sum : [ℕ] → ℕ`	adds the lengths up
6	`≤`	`≤ = length ≼ length°`	`s ≤ t` iff `length(s) ≤ length(t)`
7	`F(≤) h`	`list(≤) concat`	elementwise longer, then concat
8	`F(cost) F(≼) k cost°`	`#' ≤ +/ #°`	`#'` count each, `≤` elementwise, `+/` sum, `#°` converse of length
9	`cost : h → k`	`(length : concat → sum)` `length(concat(xs)) = sum(map(length)(xs))`	`+/#' = #,/`, i.e. `# : ,/ → +/`
10	`k Pres ≼`	`sum Pres ≤` if two lists have the same length and $n_i \le m_i$ at every position, then `sum(ns) ≤ sum(ms)`	elementwise smaller implies total smaller

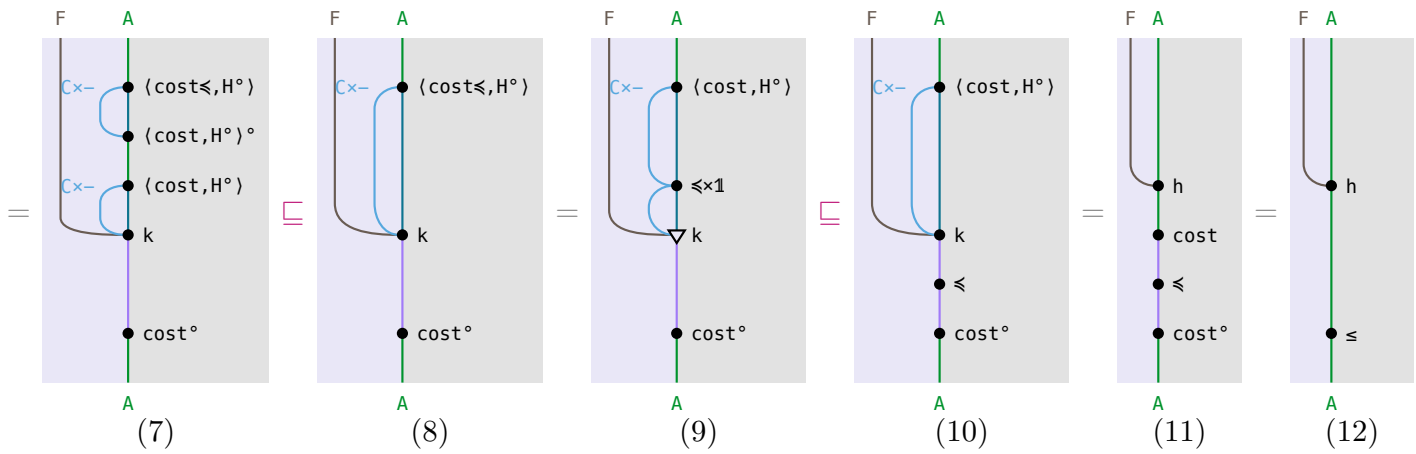
Map(cost); ≤ = cost ⩽ cost°; Simple(H°); h cost = F(⟨cost,H°⟩) k; F(⩽ × 𝟙) k ⊑ k ⩽ ⇒ F(≤ ∩ (H° H)) h ⊑ h ≤ Proposition 9.3

improving each part by ≤ within its H°-context, then assembling by h, is below h ≤



(1) (2) (3) (4) (5) (6)

1	F(≤ ∩ (H° H)) h	
2	= F(≤ ∩ (H° H)) h	Map(cost) ⇒ 𝟙 ⊑ cost cost°
3	⊑ F(≤ ∩ (H° H)) h cost cost°	H°° = H
4	= F(≤ ∩ (H° H°°)) h cost cost°	≤ = cost ⩽ cost°
5	= F((cost ⩽ cost°) ∩ (H° H°°)) h cost cost°	⟨X,Y⟩ ⟨cost,S⟩° = (X cost°) ∩ (Y S°)
6	= F(⟨cost ⩽,H°⟩ ⟨cost,H°⟩°) h cost cost°	h cost = F(⟨cost,H°⟩) k



(7) (8) (9) (10) (11) (12)

7	= F(⟨cost ⩽,H°⟩ ⟨cost,H°⟩° ⟨cost,H°⟩) k cost°	
8	⊑ F(⟨cost ⩽,H°⟩) k cost°	⟨cost,Y⟩ (⩽ × 𝟙) = ⟨cost ⩽,Y⟩
9	= F(⟨cost,H°⟩) F(⩽ × 𝟙) k cost°	F(⩽ × 𝟙) k ⊑ k ⩽
10	⊑ F(⟨cost,H°⟩) k ⩽ cost°	h cost = F(⟨cost,H°⟩) k
11	= h cost ⩽ cost°	≤ = cost ⩽ cost°
12	= h ≤	

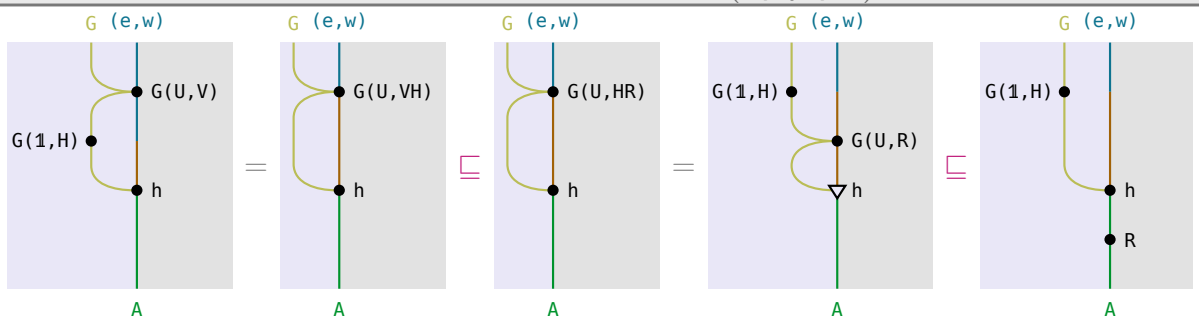
(9.1.3d)

	symbol	type	value	meaning
1	F		F(Y) = X + Y×Y	a tip holding one x, or two parts to join
2	h	F(A) → A	[tip, bin] : F(tree X) → tree X	a bracketing is a tip, or two bracketings joined
3	cost	A → C	cost : tree X → ℕ	the cost of evaluating a bracketing
4	H	B → A	flatten° : [X] → tree X	sends a list to every bracketing of it; in Theorem 9.1 H ≜ ⦇T⦈°⦇h⦈
5	H°	A → B	flatten	sends a bracketing to the one list it brackets; a map, so simple
6	⟨cost, H°⟩	A → C×B	⟨cost, flatten⟩	tags a bracketing with its cost and the list it brackets
7	⩽	C → C	≤ on ℕ	compares the costs
8	k	F(C×B) → C	g	0 at a tip; at a join, the two costs plus cb of the two sizes, and a size is read off the list (sz), so k needs the context
9	≤ ∩ H°H	A → A	≤ ∩ flatten flatten°, where ≤ = cost ⩽ cost°	t costs no more than t', and both bracket the same list

Proposition 9.4

`G(U,R) h ⊑ h R; V H ⊑ H R`
`⇒ G(U,V) G(𝟙,H) h ⊑ G(𝟙,H) h R`

thinning the parts by **`U`** in the first argument and by **`V`** in the second (**`G(U,V)`**), then solving by **`H`** and assembling by **`h`**, gives only what solving and assembling and then improving by **`R`** gives (**`G(𝟙,H)hR`**)



1	`G(U,V) G(𝟙,H) h`	
2	`= G(U,V H) h`	`V H ⊑ H R`
3	`⊑ G(U,H R) h`	
4	`= G(𝟙,H) G(U,R) h`	`G(U,R) h ⊑ h R`
5	`⊑ G(𝟙,H) h R`	

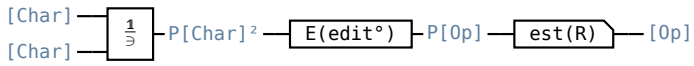
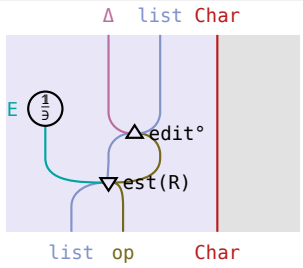
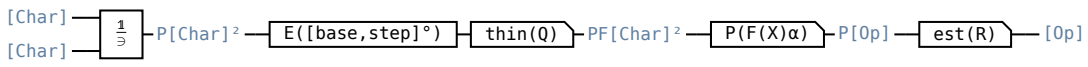
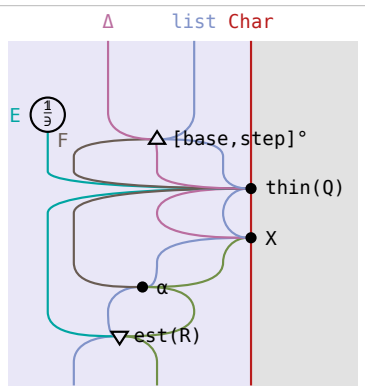
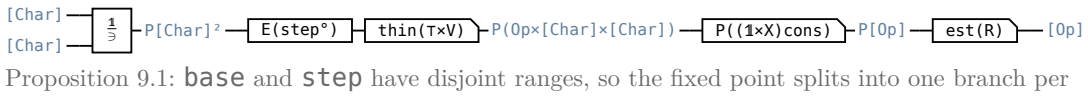
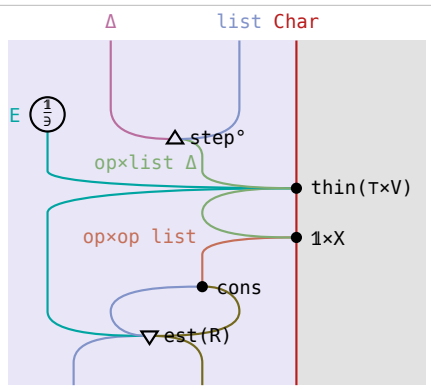
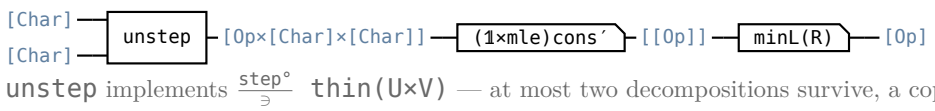
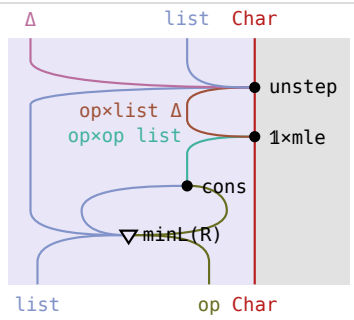
§9.2. The string edit problem

(9.2a)

	definition	type	meaning
1	`Op ::= cpy(Char) | del(Char) | ins(Char)`	`Type`	an edit operation copies, deletes or inserts one character
2	`FX = 1 + (Op × X)`		`F(Op,X)` is the empty sequence or one operation paired with an `X`
3	`edit ≜ ⦇[base,step]⦈`	`[Op]→[Char]×[Char]`	the two strings an edit sequence turns one into the other
4	`length = ⦇[zero,π₂ succ]⦈`		the number of operations in a sequence
5	`R ≜ length ≤ length°`	`[Op]→[Op]`	`es` is no longer than `fs`
6	`V ≜ suffix° × suffix°`	`[Char]×[Char]→ [Char]×[Char]`	each string of the first pair is a suffix of the matching string of the second
7	`Q ≜ F(⊤,V)`	`F([Char]×[Char])→ F([Char]×[Char])`	any two operations, and the two remaining pairs of strings compared by `V`
8	`empty(p) ≜ p = ([],[])`	`[Char]×[Char]→ [Char]×[Char]`	holds only at the pair of two empty strings
9	`α ≜ [nil,cons]`	`F([Op])→[Op]`	the empty edit sequence, or one operation in front of a sequence
10	`base = ([],[])` , `step(cpy(a),(xs,ys)) = ([a]#xs,[a]#ys)` , `step(del(a),(xs,ys)) = ([a]#xs,ys)` , `step(ins(a),(xs,ys)) = (xs,[a]#ys)`	`Unit + Op × [Char]×[Char]→ [Char]×[Char]`	`[base,step]`: the two empty strings, or one operation added to the front of the pair

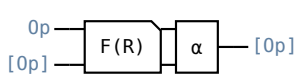
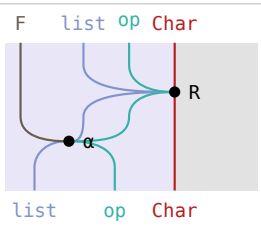
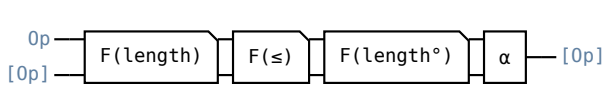
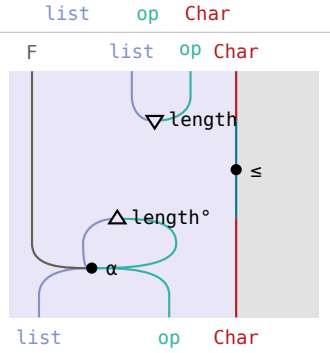
`mle=(empty→nil,unstep ((𝟙×mle)cons)´ minL(R)), (μX : [base,step]° ∋ thin(Q) P(F(X) α) est(R)) ⊑ edit° ∋ est(R)` (9.2b)

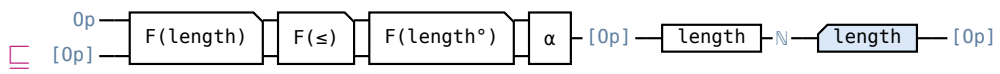
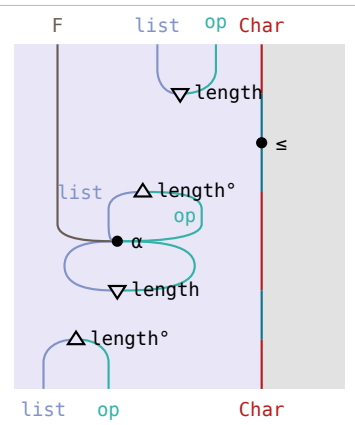
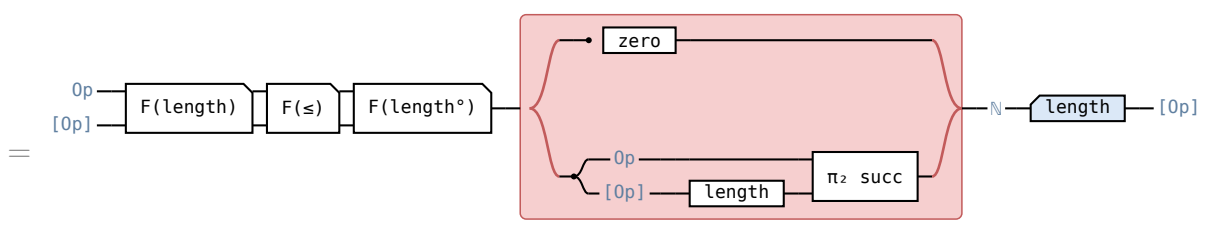
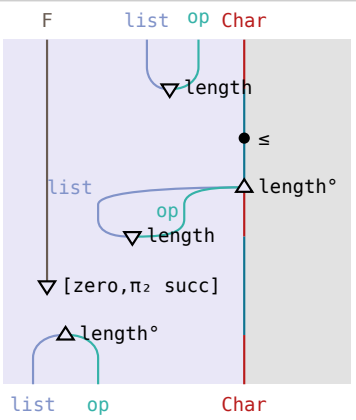
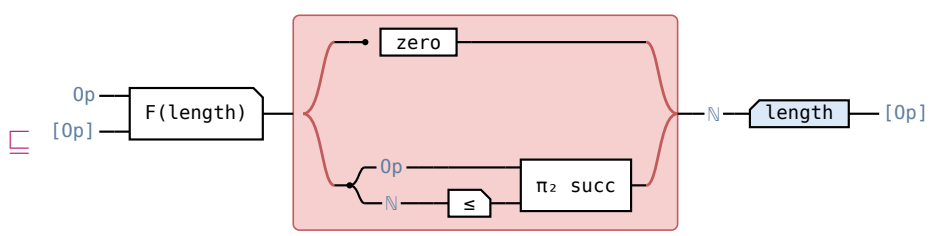
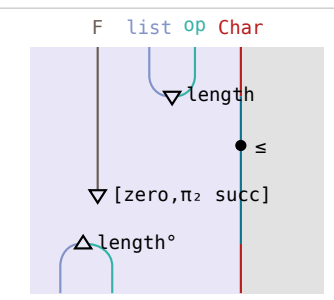
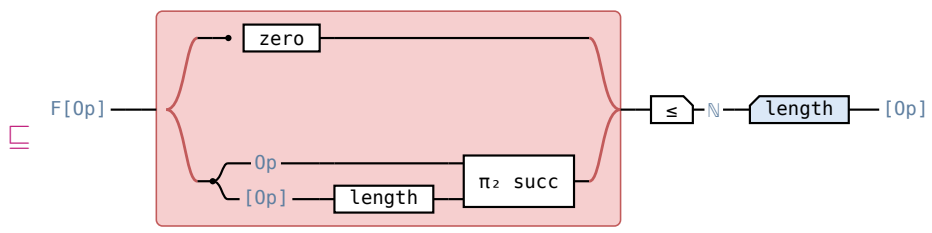
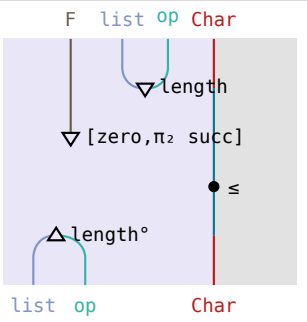
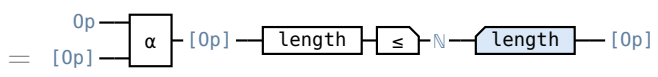
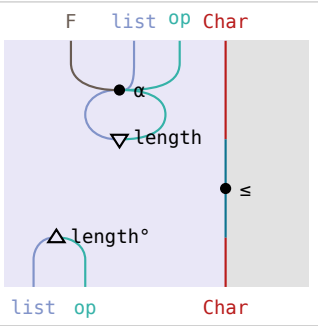
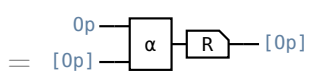
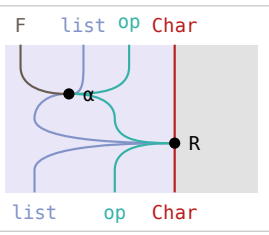
a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

circuit	Hinze–Marsden
 the specification — `edit([]) = ([],[]), edit([op]#es) = step(op,edit(es))` , `edit ≜ ⦇[base,step]⦈`	
 ⊒ at `Q≜𝟙+(U×V)`. Monotonicity `F(R)α⊑αR` is Proposition 9.2 at `length≜⦇[zero,π₂ succ]⦈` with `succ` monotonic on `≤`, so `cons` is monotonic on `R`. The thinning condition `QF(𝟙,edit°)α⊑F(𝟙,edit°)αR` over `F(Op,[Char]×[Char])` is Proposition 9.4 at `U≜⊤` — `F(⊤,R)α⊑αR`, left as an exercise, so any two operations may be compared — and `V≜suffix°×suffix°` , — `edit (𝟙×suffix)⊑R° edit` , `edit (suffix×𝟙)⊑R° edit` . `suffix=tail*` and `BA⊑CB⇒BA*⊑C*B` cut those to one step each: drop the operation that produced the head, or weaken its `cpy` to a `del`, never lengthening the sequence	
 = Proposition 9.1: `base` and `step` have disjoint ranges, so the fixed point splits into one branch per summand — `empty`, the coreflexive on `(xs,ys)` with both lists empty, is where `base` returns	
 ⊒ `unstep` implements `step° ∋ thin(U×V)` — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and `minL(R)` implements `est(R)`. The same subproblem is solved many times over, so the running time is exponential in the two lengths	

`F(R) α ⊑ α R` (9.2c)

comparing the tails by `R` and then putting an operation in front (`F(R)α`) relates only sequences that `R` also relates once the operation is in front (`αR`)

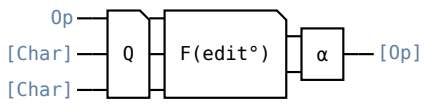
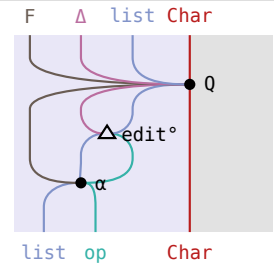
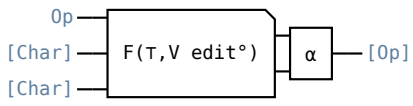
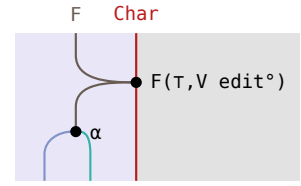
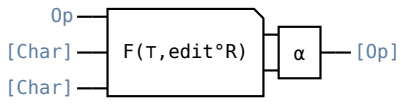
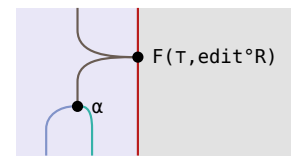
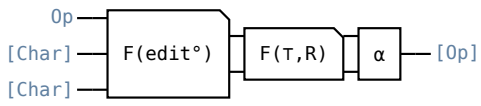
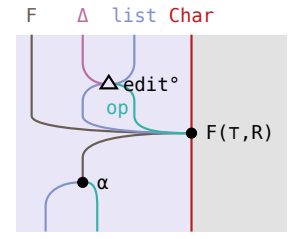
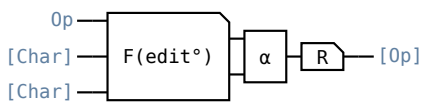
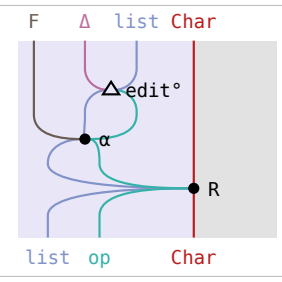
circuit	Hinze–Marsden
	
 = `R≜length≤length°` — `es R fs ⇔ length(es) ≤ length(fs)` , `R ≜ length ≤ length°` ; `F` preserves composition	

circuit	Hinze–Marsden
⊑  `length` is a map, so entire: `𝟙⊑length length°`	
=  `α length=F(length)[zero,π₂ succ]`: `length≜([zero,π₂ succ])` — `es R fs ⇔ length(es) ≤ length(fs)` , `R ≜ length ≤ length°`	
⊑  `length` is a map, so simple, and `F` is monotonic: `F(length°)F(length)⊑F(𝟙)=𝟙`	
⊑  `succ` is monotonic on `≤`: `F(≤)[zero,π₂ succ]⊑[zero,π₂ succ]≤`	
=  `F(length)[zero,π₂ succ]=α length` again	
=  `R≜length≤length°` — `es R fs ⇔ length(es) ≤ length(fs)` , `R ≜ length ≤ length°`	

Q F(edit°) α ⊑ F(edit°) α R (9.2d)

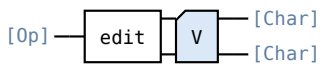
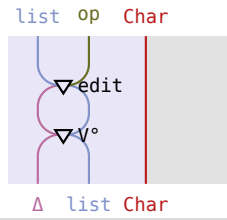
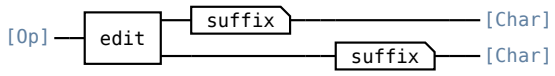
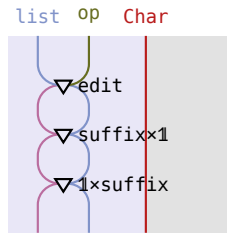
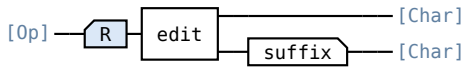
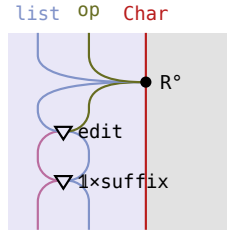
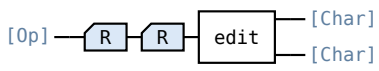
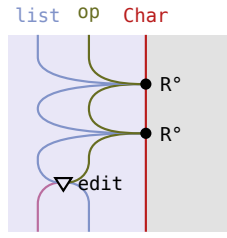
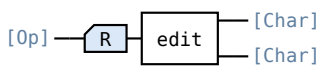
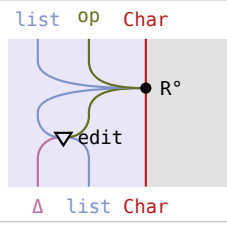
F(Op,[Char]×[Char]) —F(𝟙,edit°)→ **F(Op,[Op])** —α→ **[Op]**

Q compares here **R compares here**

circuit	Hinze–Marsden
	
= 	
⊑  Proposition 9.4's second condition `V edit°⊑edit° R` — `edit V° ⊑ R° edit`	
=  F preserves composition: `F(U,edit° R)=F(𝟙,edit°)F(U,R)`	
⊑  Proposition 9.4's first condition `F(⊤,R)α⊑αR`, left as an exercise in the book: `cons` adds one to both lengths whatever the two operations are	

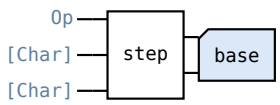
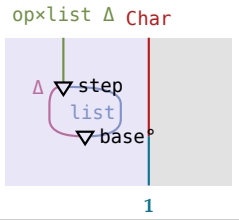
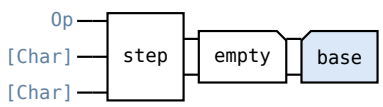
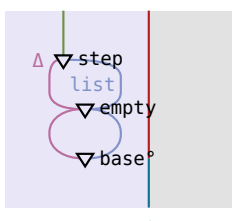
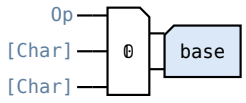
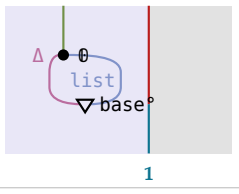
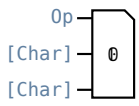
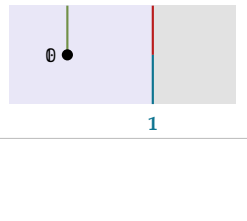
edit V° ⊑ R° edit (9.2e)

cutting a front off either string an edit sequence produces (`edit V°`) leaves a pair that a sequence no longer produces (`R° edit`)

circuit	Hinze–Marsden
	
=  `V≜suffix°×suffix°`, and `×` preserves composition	
⊑  `edit (suffix×𝟙)⊑R° edit`: drop the operation that produced the head, or weaken its `cpy` to an `ins`. Proved by induction at `suffix` directly, not through `suffix=tail*` and `BA⊑CB⇒BA*⊑C*B` as the book does	
⊑  `edit (𝟙×suffix)⊑R° edit`, the mirror image: `cpy` weakens to a `del`	
⊑  `R°` is transitive, `≤` being so	

step base° = 𝟘 (9.2f)

no pair of strings is both a result of `step` and the result `([],[])` of `base`

circuit	Hinze–Marsden
	
=  `base` returns only `([],[])`, so `base°=empty base°`	
=  `step empty=𝟘`: `cpy` and `del` put a character on the left string, `ins` one on the right	
=  𝟘 composed with anything is 𝟘	

§9.2.1. Tabulation

(9.2.1a)

```
1  mle(xs,ys) = head(column(xs,ys)) ,  column(xs,ys)=[mle(u,ys)|u←tails(xs)]
   =
   column(xs)=⦇[fstcol(xs),nextcol(xs)]⦈,  fstcol=del´ tails
   the tabulation: mle(xs,ys) needs mle(u,v) for every tail u of xs and v of ys, so the columns are built right to left

2  column(xs,[b]#ys) = nextcol(xs,(b,column(xs,ys)))
   =
   nextcol(xs)(b,us)=⦇[base(b,last(us)),step(b)]⦈(xus),  xus=zip(xs,zip(init(us),tail(us)))
   each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right

3  base(b,u)=[[ins(b)]#u]
   =
   step(b)((a,(u,v)),ws)=(a=b→[[cpy(a)]#v]#ws, [bmin(R)([del(a)]#w,[ins(b)]#u)]#ws)
   w=head(ws); these base, step are not edit's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one
   below that (a copy) — quadratic in the two lengths
```

§9.3. Optimal bracketing

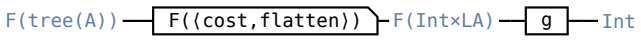
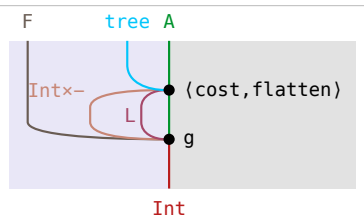
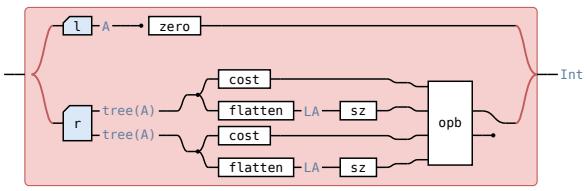
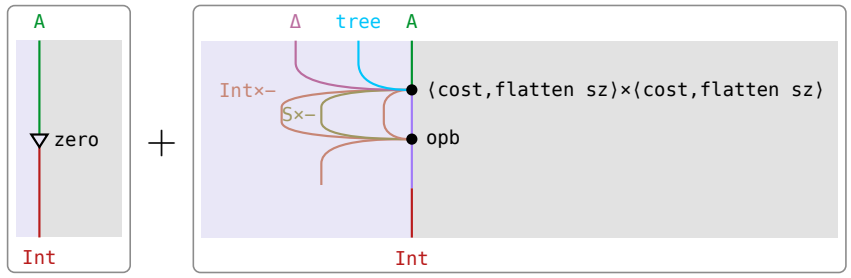
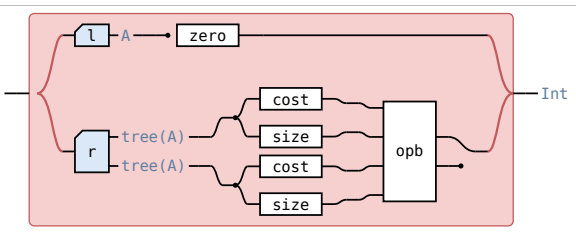
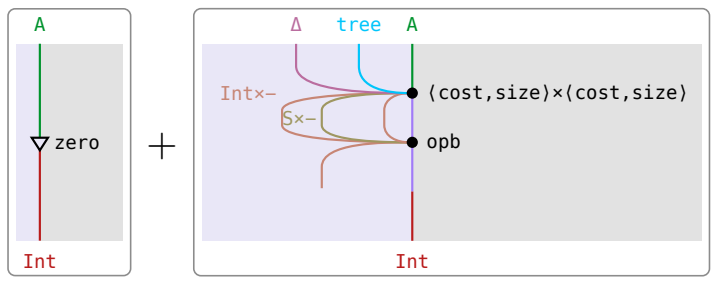
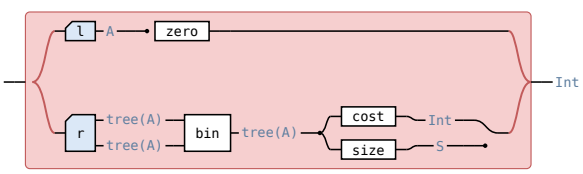
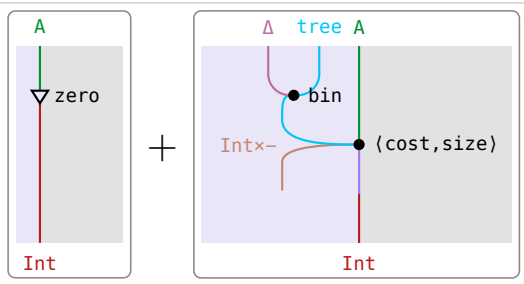
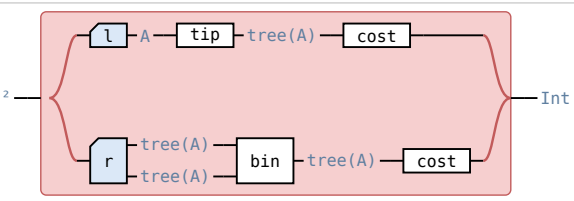
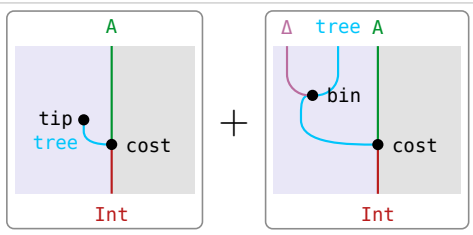
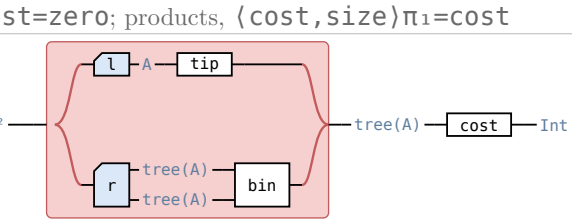
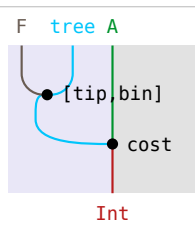
(9.3a)

	definition	type	meaning
1	`tree(A) ::= tip(A) | bin(tree(A),tree(A))`	`Type`	a tree is a tip holding one value, or a node joining two trees; `h≜[tip,bin]`
2	`⦇[τ,cat]⦈ = flatten`	`tree(A)→LA`	the tips of a tree, left to right; `H=flatten°` builds every tree over a list
3	`opb(p) ≜ ((cb(π₂(π₁(p)),π₂(π₂(p))) + π₁(π₁(p))) + π₁(π₂(p)), sb(π₂(π₁(p)),π₂(π₂(p))))`	`Int×S`	joining two subtrees costs `cb` of their sizes plus the cost of each
4	`⟨cost,size⟩(tip(a)) = (0,st(a)), ⟨cost,size⟩(bin(xs,ys)) = opb(⟨cost,size⟩(xs),⟨cost,size⟩(ys))`	`tree(A)→Int×S`	the cost of evaluating a bracketing, paired with the size of its result
5	`sz([a]) = st(a), sz([a]⧺xs) = sb(st(a),sz(xs)) , Assoc(sb) ⇒ π₂(⟨cost,size⟩(xs)) = sz(flatten(xs))`	`LA→S`	`sb` associative, so the size depends only on the flattened list
6	`R ≜ cost ≤ cost°`	`tree(A)→tree(A)`	`t` costs no more than `t'`
7	`g ≜ [⊸ 0,(𝟙×sz)² opb π₁]`	`F(Int×LA)→Int`	the cost of the top step alone: zero at a tip, `cb` of the two sizes at a node
8	`nonsingle(xs) ≜ ∃a. ∃zs. xs = [a]⧺zs`	`LA→LA`	holds at the lists of two or more elements
9	`splits ≜ ⟨inits⁺,tails⁺⟩ zip`	`LA→[L A×L A]`	every way to cut a list into two non-empty parts — an implementation of $\frac{\texttt{cat}^\circ}{\ni}$
10	`array ≜ inits row′`	`LA→[[tree A]]`	the best trees of every segment, held as rows
11	`row ≜ tails mct′`	`LA→[tree A]`	the best tree of every non-empty suffix
12	`col ≜ inits mct′`	`LA→[tree A]`	the best tree of every non-empty prefix
13	`mix ≜ zip bin′ minL(R)`	`[tree A]×[tree A]→ tree(A)`	join the matching prefix and suffix trees and keep a cheapest
14	`next ≜ ⟨π₁,mix⟩ snoc`	`[tree A]×[tree A]→ [tree A]`	add the next best tree to the end of the column
15	`process ≜ ((tip τ) × 𝟙) loop(next)`	`A×[[tree A]]→ [tree A]`	start the column at a tip and run `next` over the rows

Assoc(sb) (9.5)

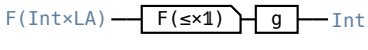
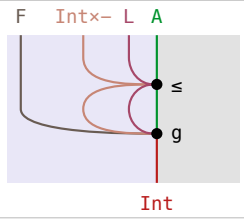
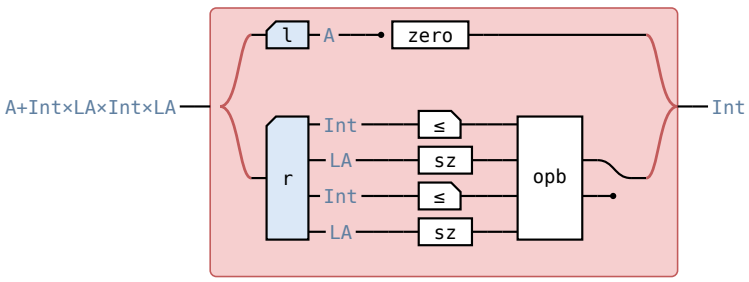
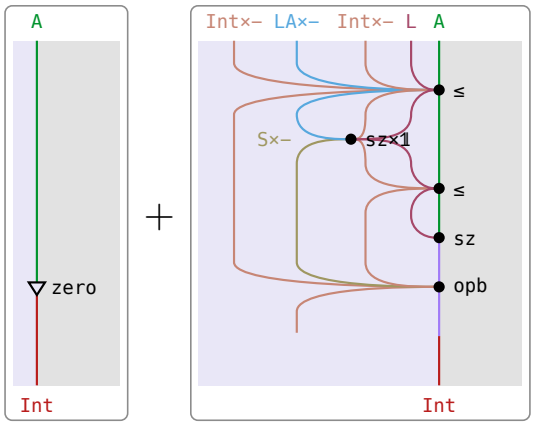
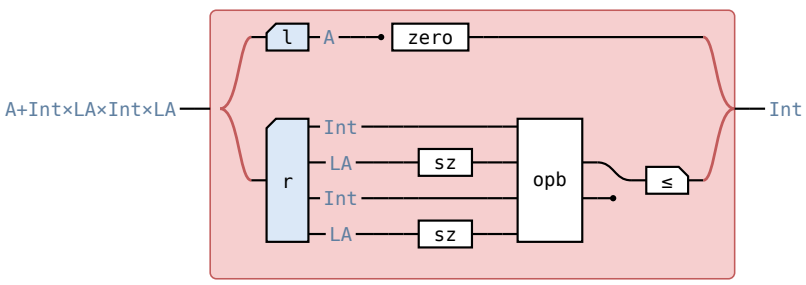
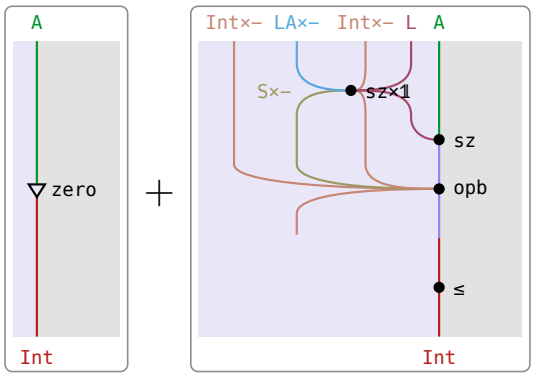
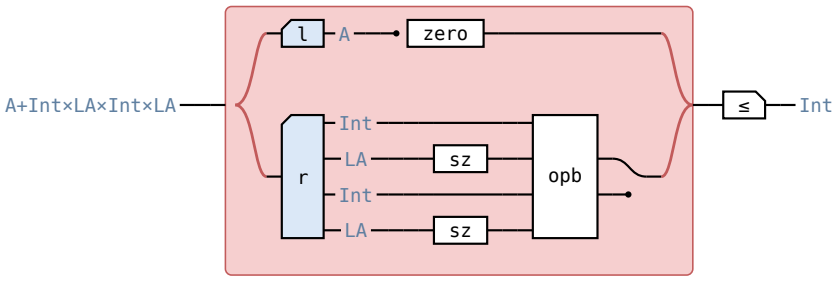
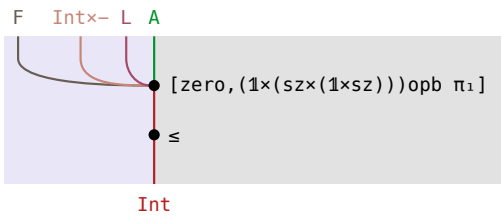
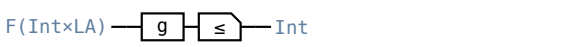
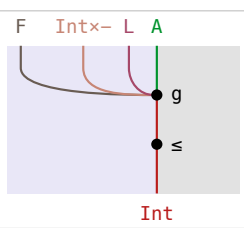
⇒ F(⟨cost,flatten⟩) g = [tip,bin] cost

pairing each subtree with its cost and its flattening and then applying **g** gives what building the tree by **[tip,bin]** and taking its **cost** gives

circuit	Hinze–Marsden
	
=  definition of g; coproducts and products — g ≜ [⊸ 0, (𝟙×sz)² opb π₁]	
=  flatten sz=size, since sb is associative — sz([a]) = st(a), sz([a]+xs) = sb(st(a),sz(xs)), Assoc(sb) ⇒ π₂((cost,size)(xs)) = sz(flatten(xs))	
=  ⟨cost,size⟩≜([opt,opb]), at a node bin⟨cost,size⟩=⟨cost,size⟩² opb	
=  tip cost=zero; products, ⟨cost,size⟩π₁=cost	
=  coproducts, [tip cost,bin cost]=[tip,bin] cost	

F(≤ × 𝟙) g ⊑ g ≤ (9.6)

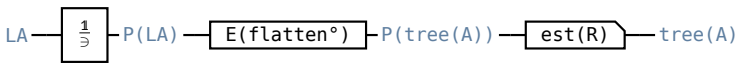
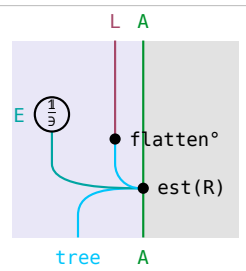
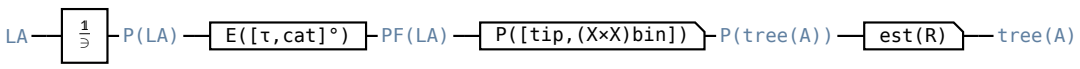
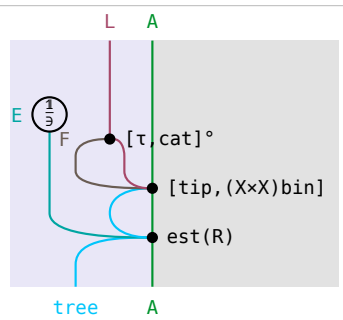
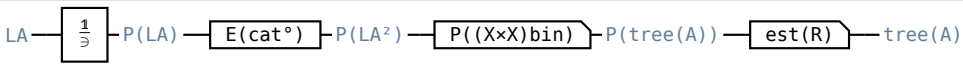
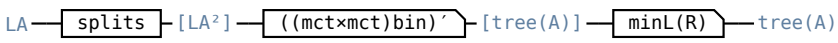
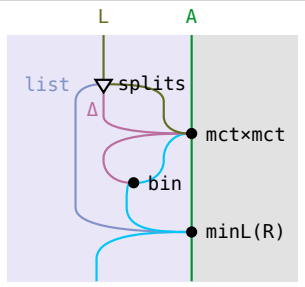
raising the cost of either subtree (≤×𝟙 in both slots) and then applying g returns a value at least what g returns on the costs before raising (g≤)

circuit	Hinze–Marsden
	
=  definition of g — g ≜ [⊸ 0,(𝟙×sz)² opb π₁]	
⊑  definition of opb, and + monotonic	
⊑  ≤ reflexive, so zero⊑zero ≤; coproducts	
=  definition of g — g ≜ [⊸ 0,(𝟙×sz)² opb π₁]	

mct=(single→head tip,⟨init col,tail row⟩ mix), Assoc(sb) (9.3b)

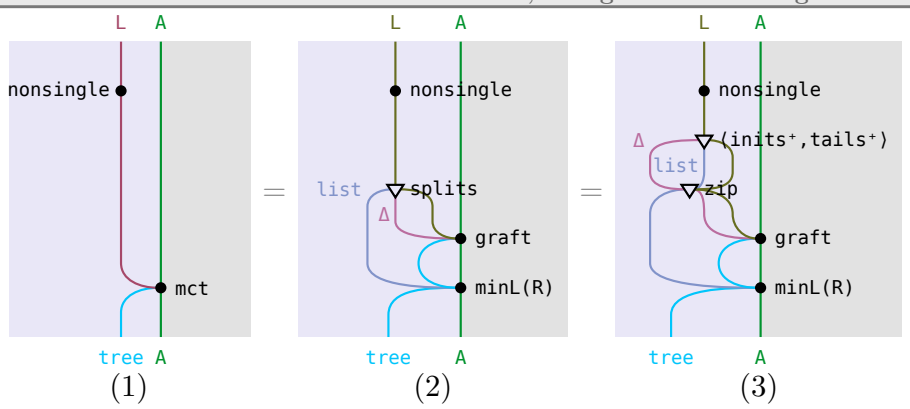
⇒ (μX : $\frac{[\tau,\mathrm{cat}]^\circ}{\ni}$ P([tip,(X × X) bin]) est(R)) ⊑ $\frac{\mathrm{flatten}^\circ}{\ni}$ est(R)

a least-cost bracketing of $a_1 \oplus \cdots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit	Hinze–Marsden
 the specification — `flatten(tip(a)) = [a], flatten(bin(xs,ys)) = cat(flatten(xs),flatten(ys))` , `(|[τ,cat]|) = flatten`	
 split the list in every way, bracket both halves, join. The condition is monotonicity **in context**, ⊒ `F(Rn(flatten flatten°))h⊑hR` — only trees with the same flattening are compared — which is Proposition 9.3 at `H°=flatten`, a map, with (9.5) `[tip,bin] cost=(𝟙+⟨cost,flatten⟩²)g` (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) `(𝟙+(≤×𝟙)²)g⊑g≤` (`g` monotonic on ≤ in its two cost arguments)	
=  Proposition 9.1: `τ` and `cat` have disjoint ranges, and `single` is the coreflexive on singleton lists, where `τ` returns	
 ⊒ `splits≜⟨inits⁺,tails⁺⟩ zip` implements $\frac{\mathrm{cat}^\circ}{\ni}$ and `minL(R)` implements `est(R)`. Exponential, since the segments of one list overlap	
= `mct=(single→head tip,⟨init col,tail row⟩ mix)` (9.7) the tabulation: `mct xs` is needed for every non-empty segment `xs`, so the values are held as an array of rows, `array≜inits row´`, `row≜tails mct´`, `col≜inits mct´`, `mix≜zip bin´ minL(R)`	
= `col=(single→head tip τ,⟨init col,tail row⟩ next)` (9.8) `cons col=(𝟙×array) process` (9.9) `row=(single→head tip τ,⟨mct,tail row⟩ cons)` (9.10) a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, `next≜⟨π₁,mix⟩ snoc`, `process≜((tip τ)×𝟙) loop(next)`	
= `array=(|[fstcol,addcol]|), fstcol≜tip τ τ` `addcol≜⟨π₁ tip τ,step⟩ cons, step≜⟨process tail,π₂⟩ zip cons´` the program: one fold building the array column by column, cubic in the length of the input	

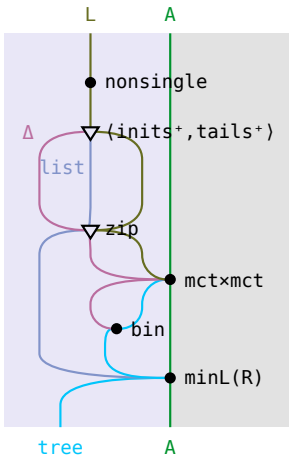
nonsingle mct = nonsingle ⟨init col,tail row⟩ mix (9.7)

on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



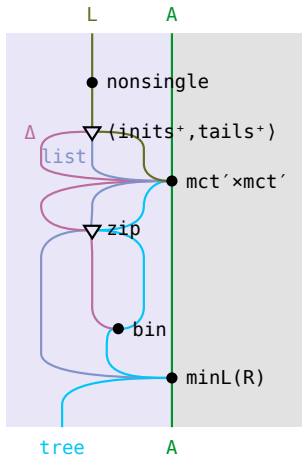
1	`nonsingle mct`	`nonsingle mct = nonsingle splits graft´ minL(R)`
2	`= nonsingle splits graft´ minL(R)`	`splits = ⟨inits⁺,tails⁺⟩ zip`
3	`= nonsingle ⟨inits⁺,tails⁺⟩ zip graft´ minL(R)`	`graft ≜ (mct × mct) bin`

≜



(4)

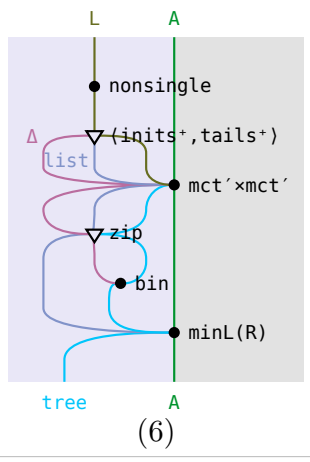
=



(5)

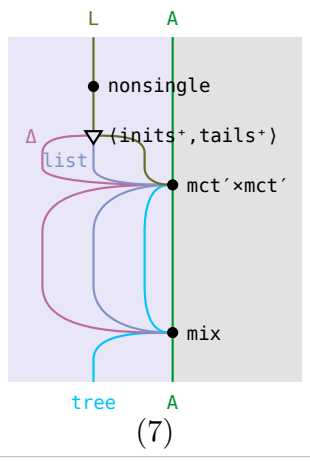
4	≜ `nonsingle ⟨inits⁺,tails⁺⟩ zip ((mct × mct) bin)′ minL(R)`	`zip (mct × mct)′ M = (mct′ × mct′) zip M`
5	= `nonsingle ⟨inits⁺,tails⁺⟩ (mct′ × mct′) zip bin′ minL(R)`	`⟨inits⁺,tails⁺⟩ (U × V) = ⟨(inits⁺) U,(tails⁺) V⟩`

=



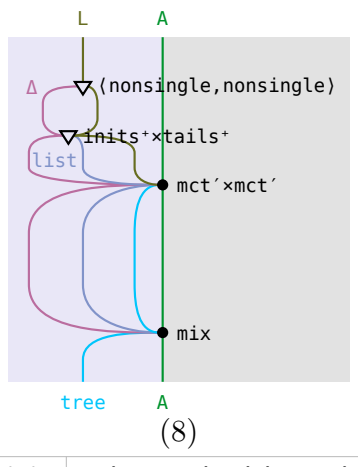
(6)

=



(7)

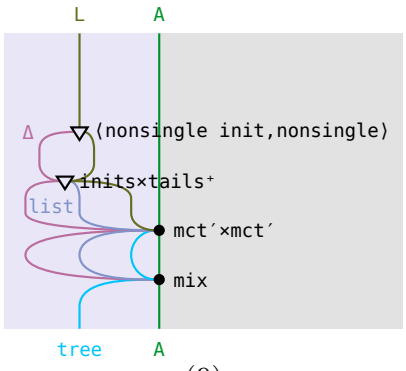
=



(8)

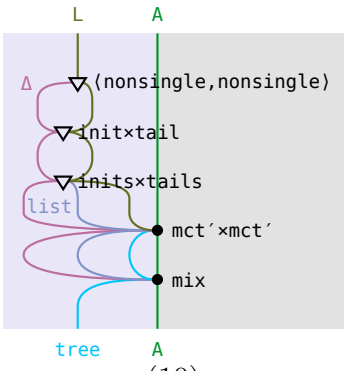
6	= `nonsingle ⟨inits⁺ mct′,tails⁺ mct′⟩ zip bin′ minL(R)`	`mix = zip bin′ minL(R)`
7	= `nonsingle ⟨inits⁺ mct′,tails⁺ mct′⟩ mix`	`nonsingle ⟨U,V⟩ = ⟨nonsingle U,nonsingle V⟩`
8	= `⟨nonsingle inits⁺ mct′,nonsingle tails⁺ mct′⟩ mix`	`nonsingle inits⁺ M = nonsingle init inits M`

=



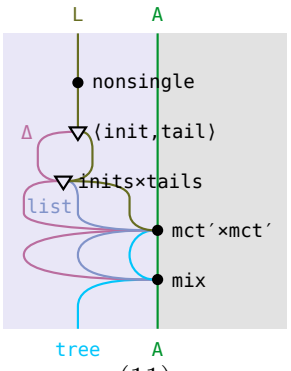
(9)

=



(10)

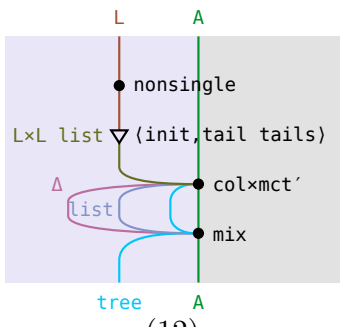
=



(11)

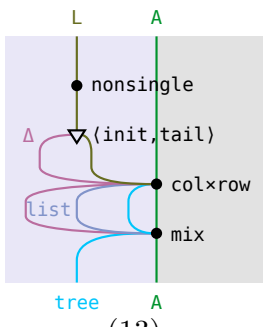
9	= `⟨nonsingle init inits mct′,nonsingle tails⁺ mct′⟩ mix`	`nonsingle tails⁺ M = nonsingle tail tails M`
10	= `⟨nonsingle init inits mct′,nonsingle tail tails mct′⟩ mix`	`nonsingle ⟨U,V⟩ = ⟨nonsingle U,nonsingle V⟩`
11	= `nonsingle ⟨init inits mct′,tail tails mct′⟩ mix`	`col = inits mct′`

=



(12)

=

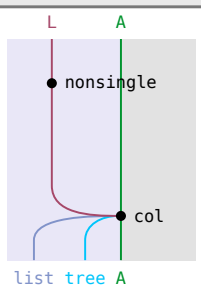


(13)

12	= `nonsingle ⟨init col,tail tails mct′⟩ mix`	`row = tails mct′`
13	= `nonsingle ⟨init col,tail row⟩ mix`	

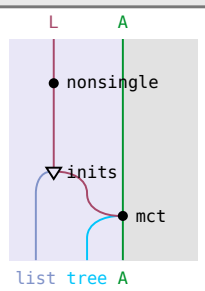
nonsingle col = nonsingle ⟨init col,tail row⟩ next (9.8)

on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**



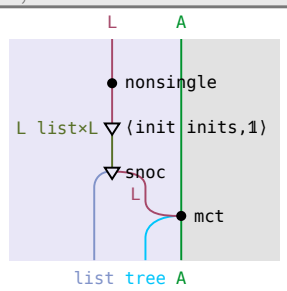
(1)

≜



(2)

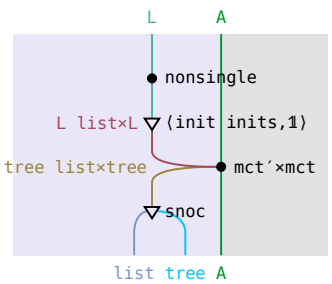
=



(3)

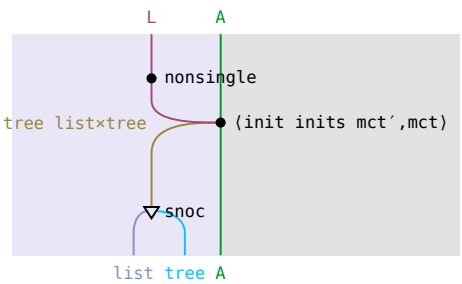
1	nonsingle col	col ≜ inits mct′
2	≜ nonsingle inits mct′	nonsingle inits M = nonsingle ⟨init inits,𝟙⟩ snoc M
3	= nonsingle ⟨init inits,𝟙⟩ snoc mct′	snoc mct′ = (mct′ × mct) snoc

=



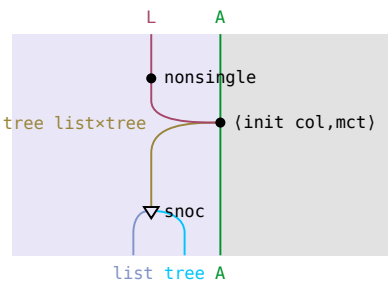
(4)

=



(5)

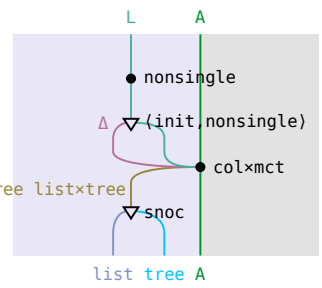
=



(6)

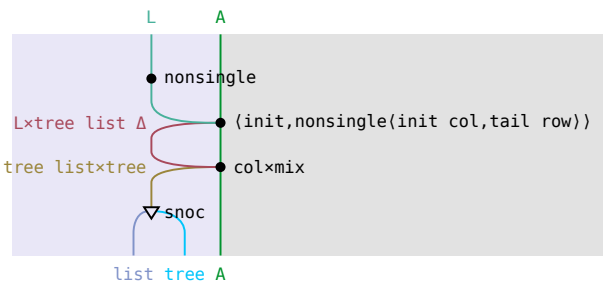
4	= nonsingle ⟨init inits,𝟙⟩ (mct′ × mct) snoc	⟨R,S⟩ (U × mct) = ⟨R U,S mct⟩
5	= nonsingle ⟨init inits mct′,mct⟩ snoc	col = inits mct′
6	= nonsingle ⟨init col,mct⟩ snoc	nonsingle ⟨U,mct⟩ = nonsingle ⟨U,nonsingle mct⟩

=



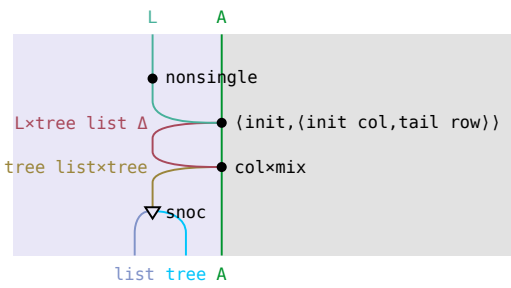
(7)

=



(8)

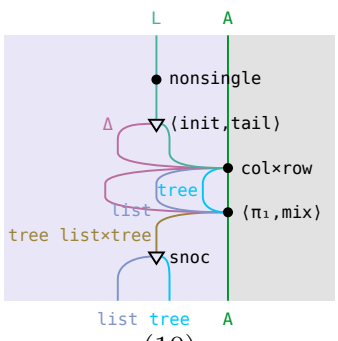
=



(9)

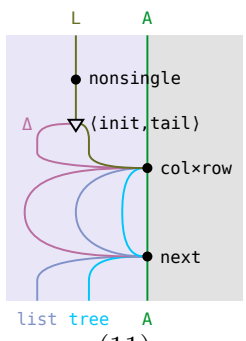
7	= nonsingle ⟨init col,nonsingle mct⟩ snoc	(9.7) nonsingle mct = nonsingle ⟨init col,tail row⟩ mix
8	= nonsingle ⟨init col,nonsingle ⟨init col,tail row⟩ mix⟩ snoc	nonsingle ⟨U,V⟩ = nonsingle ⟨U,nonsingle V⟩
9	= nonsingle ⟨init col,⟨init col,tail row⟩ mix⟩ snoc	⟨init col,V⟩ ⟨π₁,mix⟩ = ⟨init col,⟨init col,V⟩ mix⟩

=



(10)

=

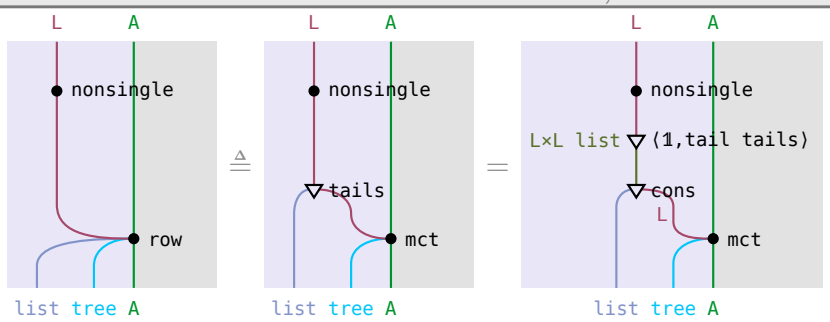


(11)

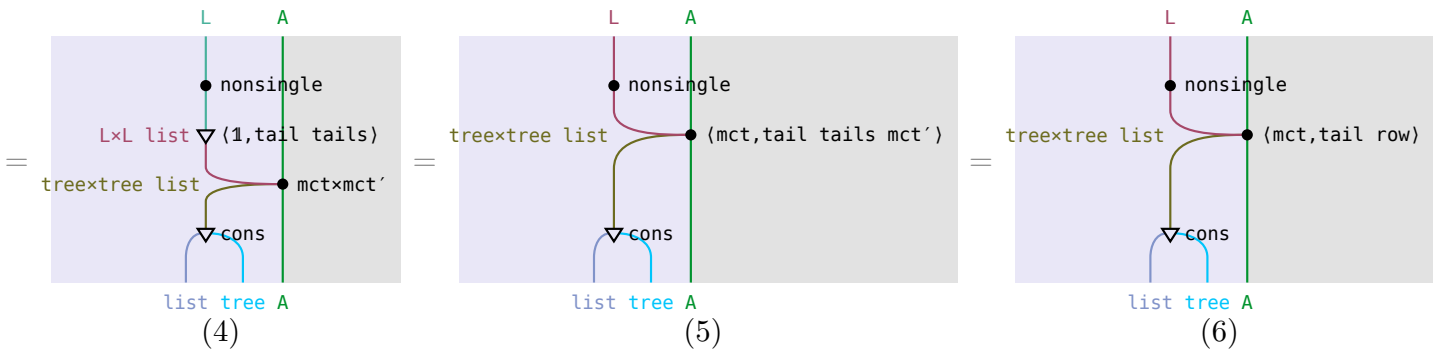
10	= nonsingle ⟨init col,tail row⟩ ⟨π₁,mix⟩ snoc	next = ⟨π₁,mix⟩ snoc
11	= nonsingle ⟨init col,tail row⟩ next	

nonsingle row = nonsingle (mct,tail row) cons (9.10)

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**



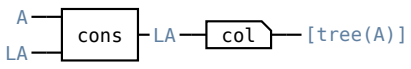
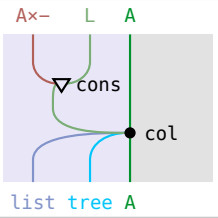
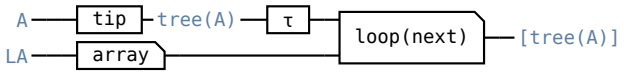
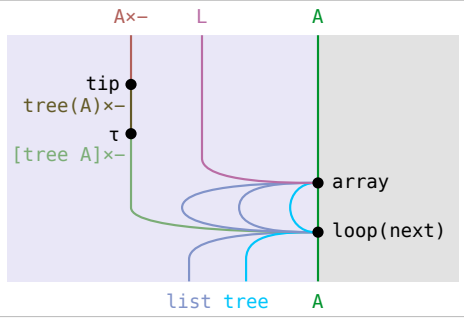
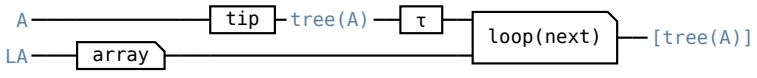
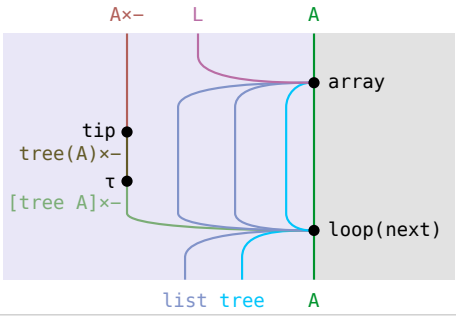
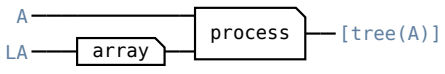
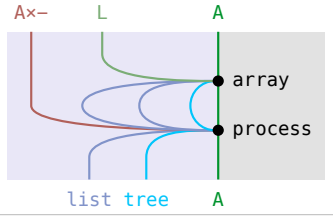
1	`nonsingle row`	`row ≜ tails mct´`
2	`≜ nonsingle tails mct´`	`nonsingle tails M = nonsingle ⟨1,tail tails⟩ cons M`
3	`= nonsingle ⟨1,tail tails⟩ cons mct´`	`cons mct´ = (mct × mct´) cons`



4	`= nonsingle ⟨1,tail tails⟩ (mct × mct´) cons`	`⟨R,S⟩ (mct × V) = ⟨R mct,S V⟩`
5	`= nonsingle ⟨mct,tail tails mct´⟩ cons`	`row = tails mct´`
6	`= nonsingle ⟨mct,tail row⟩ cons`	

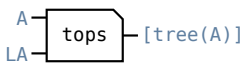
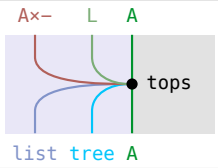
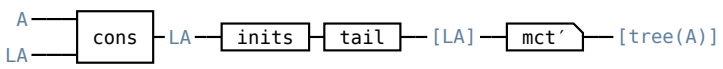
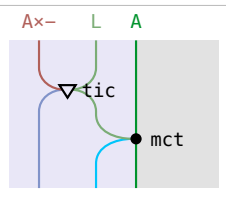
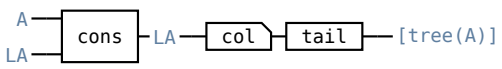
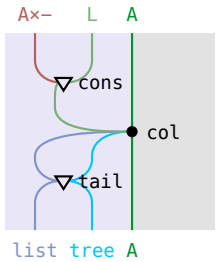
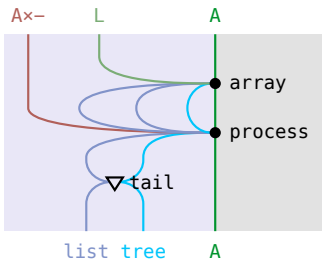
cons col = (1 × array) process (9.9)

the column of **a** consed onto **x** is what **process** makes of **a** and the array of **x**

circuit	Hinze–Marsden
	
 = Exercise 9.13 with `k≜cons col`, `g≜tip τ`, `h≜row`, `f≜next`: its two equations are (9.8) — (9.8) `nonsingle col = nonsingle ⟨init col,tail row⟩ next` — at `a:[b]` and at `a:(u++[b])`, with `[a] col=[tip(a)]`	
 = products	
 = definition of `process`	

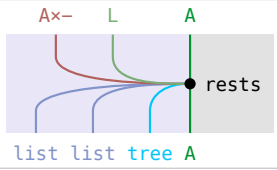
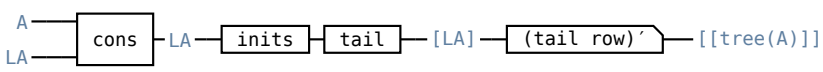
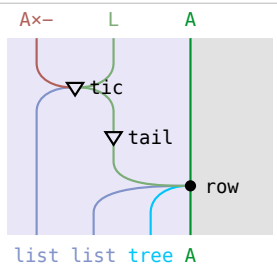
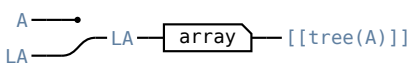
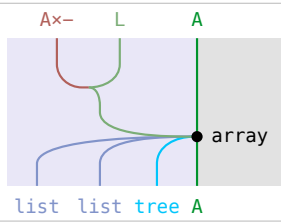
tops = (𝟙 × array) process tail (9.3c)

the trees at the top of the new rows are what **process** makes of **a** and the array of **x**, less its first

circuit	Hinze–Marsden
	
=  definition of tops≜tic mct′, our name	
=  tail f′=f′ tail; definition of col	
=  (9.9) — (9.9) cons col = (𝟙 × array) process	

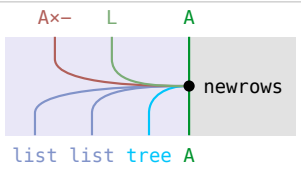
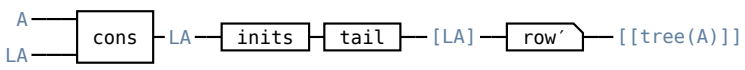
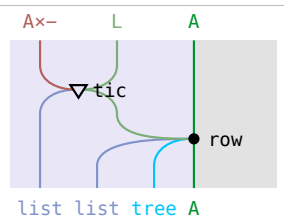
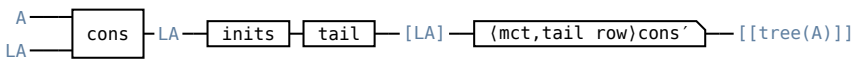
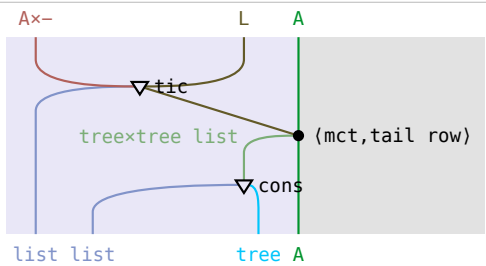
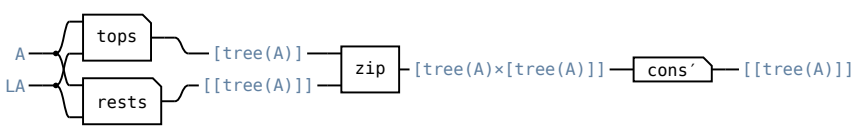
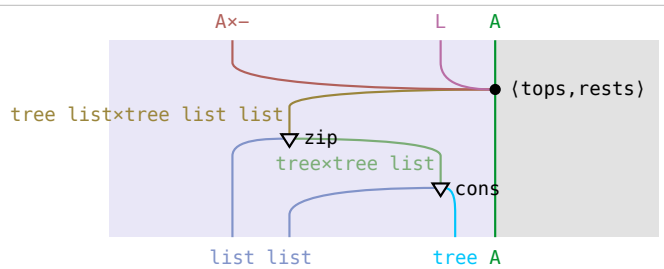
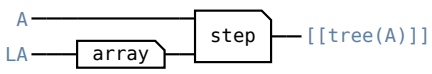
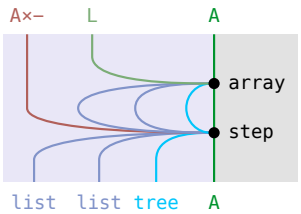
rests = π₂ array (9.3d)

the rest of each new row is the array of **x**

circuit	Hinze–Marsden
	
=  definition of rests≜tic (tail row)′, our name	
=  tic tail′=π₂ inits; definition of array	

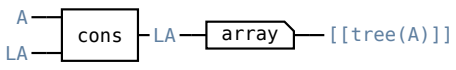
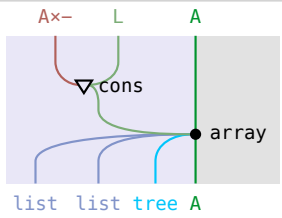
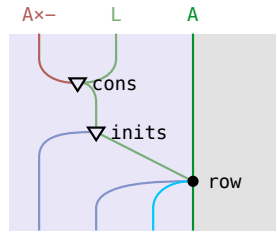
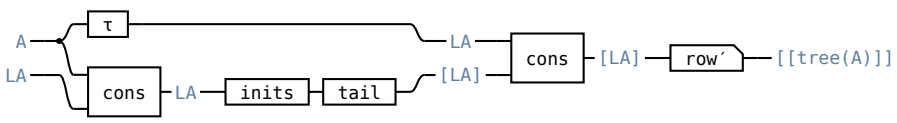
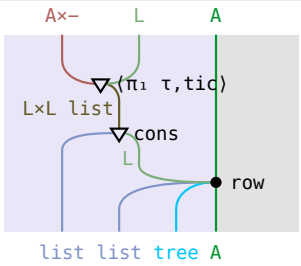
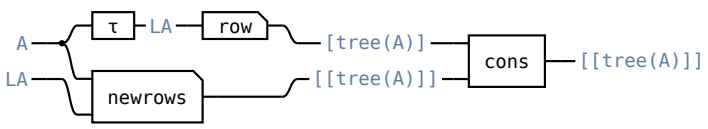
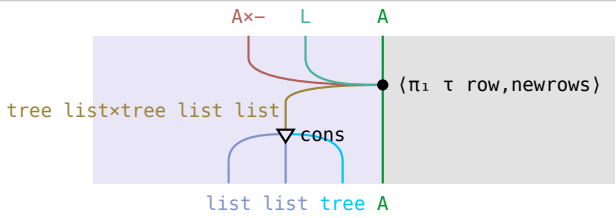
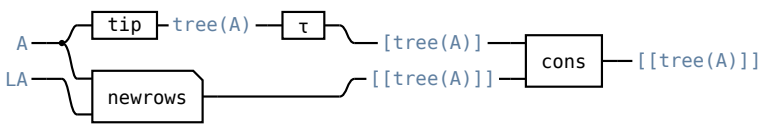
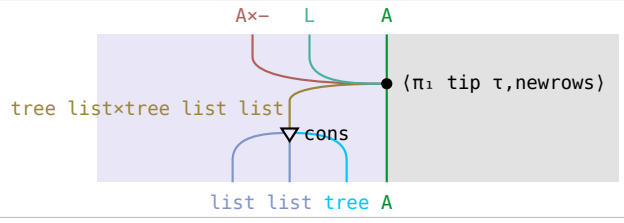
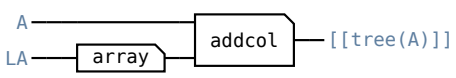
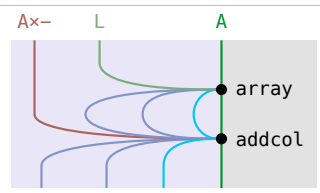
newrows = (𝟙 × array) step (9.3e)

the rows of the array of `a:x` below its first are what `step` makes of `a` and the array of `x`

circuit	Hinze–Marsden
	
=  definition of `newrows≜tic row′`, our name	
=  (9.10) — (9.10) `nonsingle row = nonsingle ⟨mct,tail row⟩ cons` — on the non-singleton `tic` lists	
=  `⟨f,g⟩′=⟨f′,g′⟩ zip`; products; definitions of `tops` and `rests`	
=  `tops = (𝟙 × array) process tail` and `rests = π₂ array`; products; definition of `step`	

cons array = (𝟙 × array) addcol (9.3f)

the array of **a** consed onto **x** is what **addcol** makes of **a** and the array of **x**

circuit	Hinze–Marsden
	
=  definition of `array`	
=  `cons inits=⟨π₁ τ,tic⟩ cons`, abbreviating `cons inits tail` by `tic`	
=  `cons f′=(f×f′) cons`; definition of `newrows`	
=  `τ row=tip τ`	
=  `newrows = (𝟙 × array) step` ; products; definition of `addcol`	

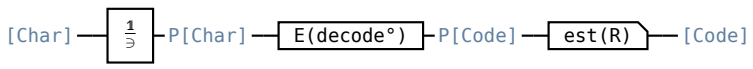
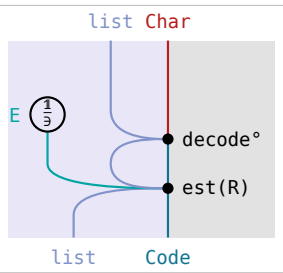
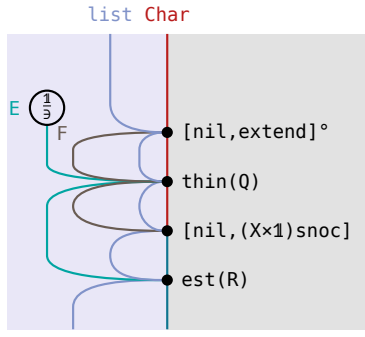
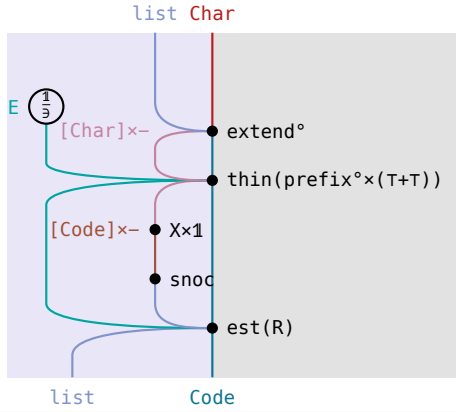
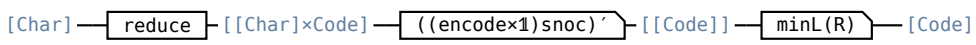
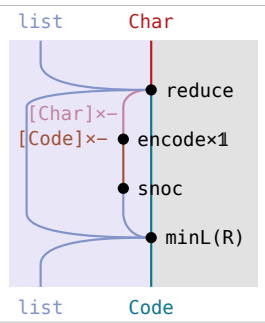
§9.4. Data compression

(9.4a)

	definition	type	meaning
1	`String ≜ [Char]`	`Type`	a string is a snoc-list of characters
2	`LA ≜ [A]`	`Type`	a non-empty snoc-list: one value, then any number more appended
3	`Code ::= sym(Char) | ptr(String,String)`	`Type`	a code is one character, or a pointer back to a repeated piece
4	`extend((xs,sym(a)),x) = x = snoc(xs,a), extend((xs,ptr(ys,zs)),x) = x = cat(xs,zs) ∧ zs ≠ [] ∧ (init⁺)(cat(ys,zs),cat(xs,zs))`	`String×Code→ [Char]`	a symbol appends its character; a pointer appends `zs` when `ys#zs` is a proper prefix of `xs#zs`
5	`decode ≜ ⦇[nil,extend]⦈`	`[Code]→[Char]`	rebuild the string from its codes, left to right; `H=decode°`
6	`bytes(sym(a)) = c, bytes(ptr(a,a')) = p`	`Code↛ℕ`	a symbol costs `c`, a pointer costs `p`
7	`⦇[nil,distr [𝟙×c,𝟙×p] plus]⦈ = size`	`[Code]→ℕ`	the total cost of a code sequence
8	`R ≜ size ≤ size°`	`[Code]→[Code]`	`u` costs no more than `v`
9	`prefix ≜ cat° π₁`	`[Char]→[Char]`	split the string in two and keep the first part
10	`(T+T) ≜ [sym,ptr]° (T + T) [sym,ptr]`	`Code→Code`	any symbol to any symbol, any pointer to any pointer
11	`Q ≜ F((T+T),prefix°)`	`F[Char]→ F[Char]`	compare two decompositions by the prefix order on the strings and `U` on the codes
12	`q ∈ reduce(ws) ⇔ extend(q,ws)`	`[Char]→ [String×Code]`	the last code of a string: one symbol, or a pointer to its longest repeated tail

(μX : [nil,extend]° thin(Q) P([nil,(X × 𝟙) snoc]) est(R)) ⊑ decode° est(R) , (9.4b)
encode=(null→nil,reduce ((encode×𝟙)snoc)′ minL(R))

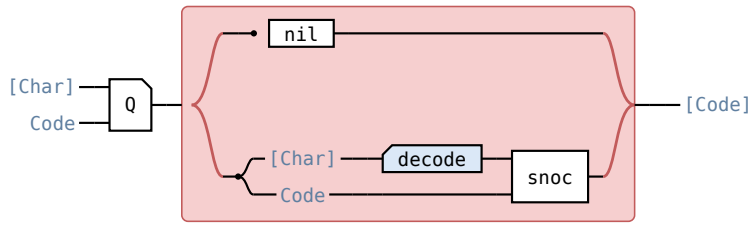
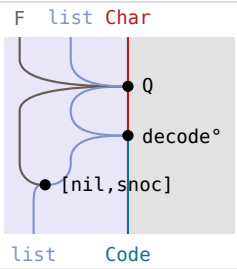
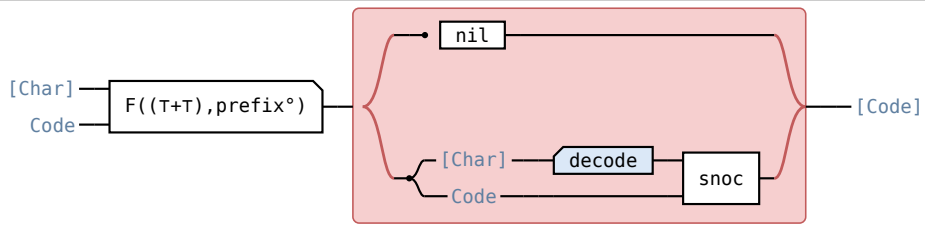
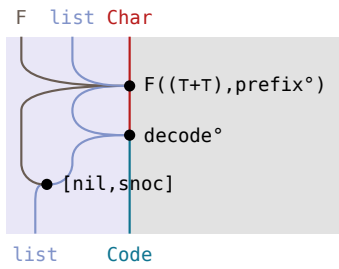
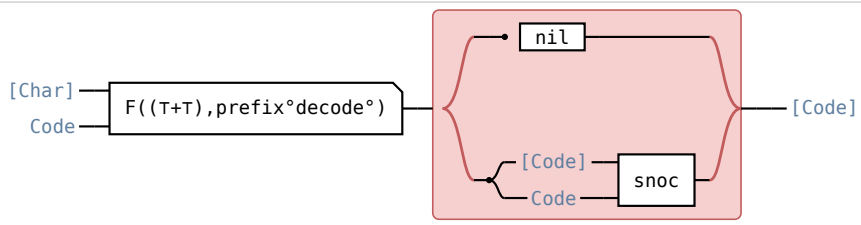
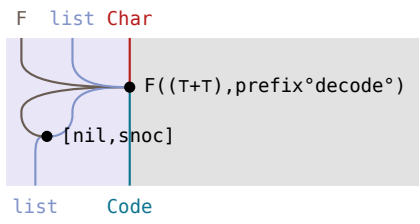
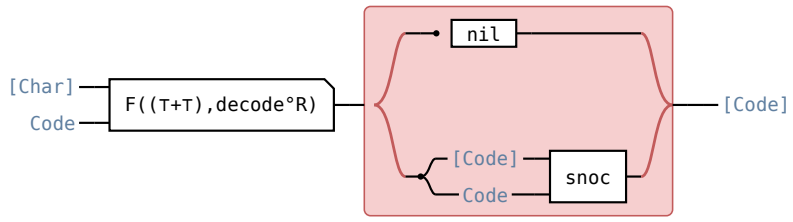
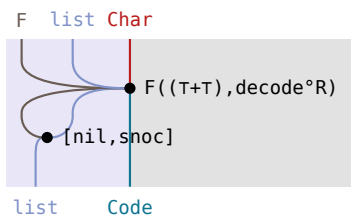
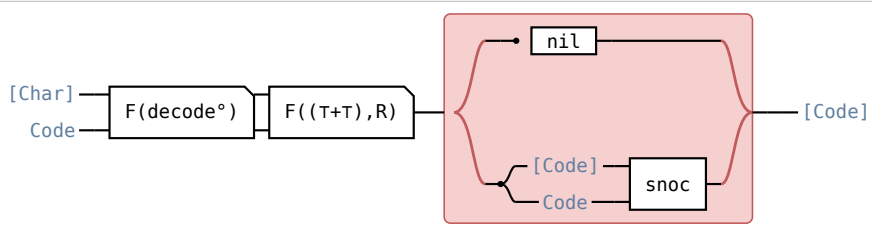
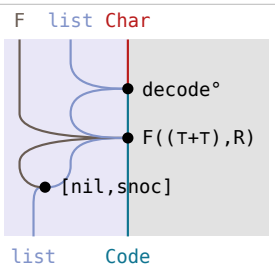
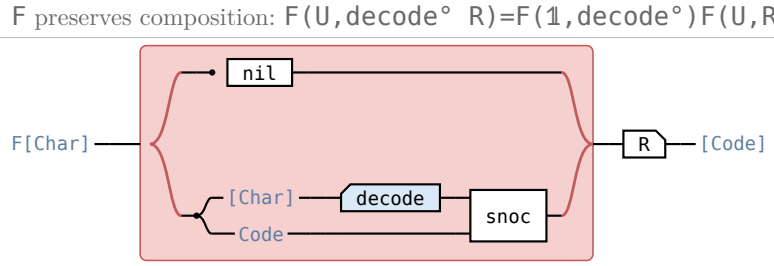
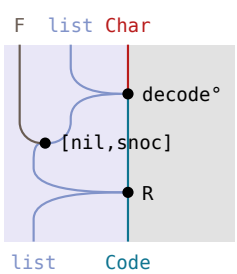
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit	Hinze–Marsden
 the specification — `decode ≜ ⦇[nil,extend]⦈`	
 at `Q≜F(T+T,prefix°)=𝟙+(prefix°×(T+T))`, the two `T` on symbols and on pointers. Monotonicity `F(R)α⊑αR` is routine, the two costs being constants: `(T+T)[c,p]=[c,p]`. ⊒ Proposition 9.4 splits the thinning condition in two — `F(T+T,R)α⊑αR`, and `decode prefix⊑R° decode`, for which `decode init⊑R° decode` is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins	
 = Proposition 9.1: `nil` and `extend` have disjoint ranges. The decompositions of one string are `extend° (ws#[a])={(ws,sym a)} ∪ {(xs,ptr (ys,zs))|xs#zs=ws#[a], ys#zs` a proper prefix of `ws}` — take the last character as a symbol, or end with a pointer	
 `reduce` implements `extend° thin(prefix°×(T+T))`: thinning leaves at most two, the symbol and the pointer of the longest repeated tail `lrt`. Again exponential; the book gives no tabulation for it	

§9.4.1. Derivation

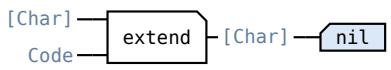
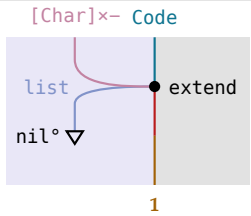
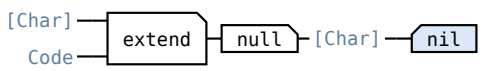
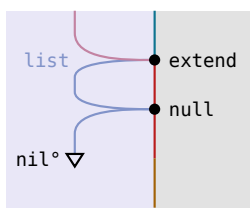
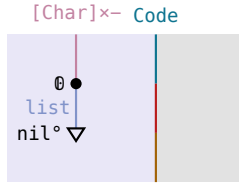
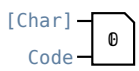
Q F(decode°) [nil,snoc] ⊑ F(decode°) [nil,snoc] R (9.4.1a)

every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one

circuit	Hinze–Marsden
	
=  Q≜𝟙+(prefix°×(T+T)) is F(T+T,prefix°) — l(val) Q l(val') ⇔ true, r(p) Q r(q) ⇔ (prefix°)(π₁(p),π₁(q)) ∧ (T+T)(π₂(p),π₂(q)), (∀val. ∀val'. u = l(val) ⇒ v = l(val') ⇒ false); (∀p. ∀q. u = r(p) ⇒ v = r(q) ⇒ false) ⇒ (u Q v ⇔ false) , Q ≜ F((T+T),prefix°)	
=  the bifunctor F preserves composition: F(U,prefix°)F(𝟙,decode°)=F(U,prefix° decode°)	
⊑  Proposition 9.4′s second condition prefix° decode°⊑decode° R: dropping the last character drops the last sym, or shortens or drops the last pointer. Proved at prefix directly by induction, not through init	
=  F preserves composition: F(U,decode° R)=F(𝟙,decode°)F(U,R)	
⊑  Proposition 9.4′s first condition F(T+T,R)α⊑αR, left as an exercise in the book: snoc adds the cost of the last element to both sides, and [c,p](T+T)=[c,p]	

extend nil° = 0 (9.4.1b)

no string is both a result of **extend** and the result **[]** of **nil**

circuit	Hinze–Marsden
	
=  `nil` returns only `[]`, so `nil°=null nil°`	
=  `extend null=0`: a symbol ends the string with a character, a pointer with its non-empty `zs`	
=  `0` composed with anything is `0`	